

2019

USB3.0 系统开发

Royal 板卡系统开发文档

本文档主要介绍板卡系统连接框架，板卡系统架构逻辑定义，参数结构解析和 SDK 接口调用等



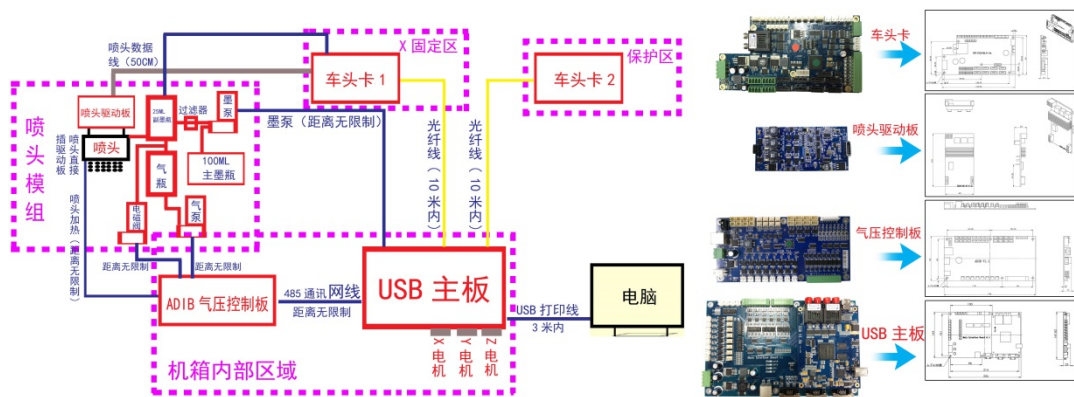
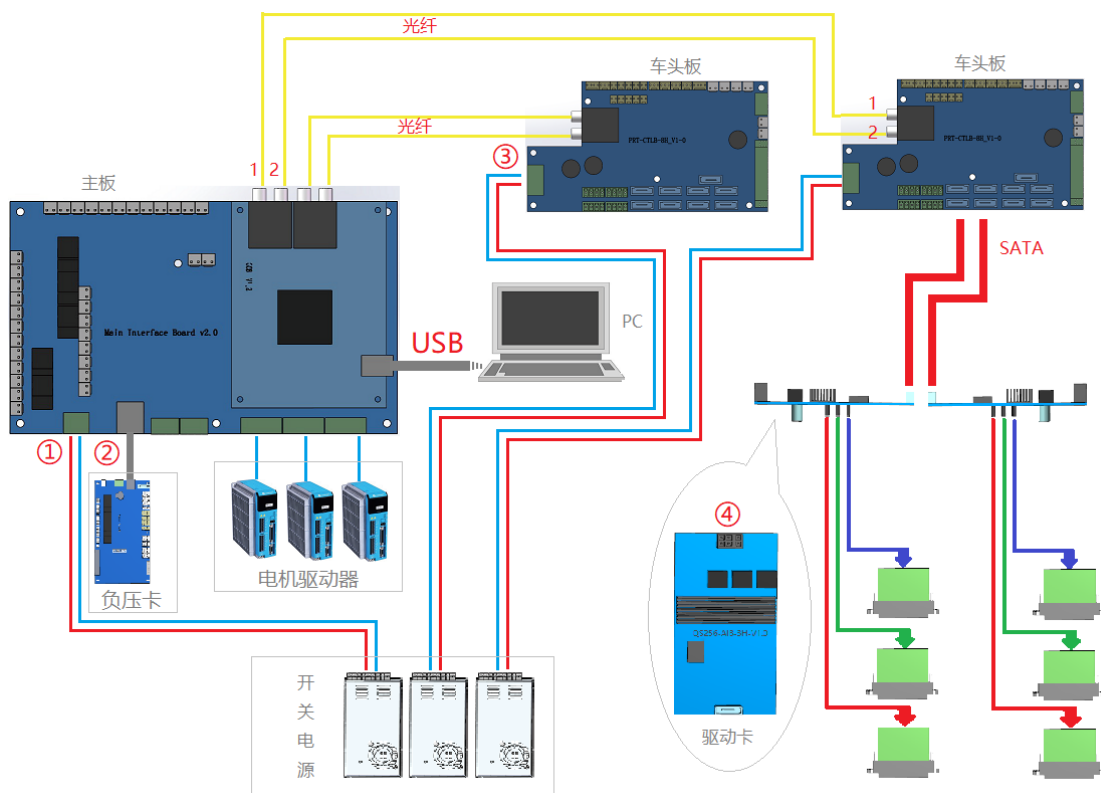
一、硬件系统连接	4
1.1 板卡连接示意图	4
1.1.1 端口说明	4
1.2 板卡端口介绍	5
1.2.1 主板	5
1.2.2 车头板	7
1.2.3 驱动卡	8
1.2.3 负压卡	9
二、喷头模组定义	10
2.1 一卡一色	10
2.2 一卡三色	10
三、坐标轴的定义	11
四、系统卡 ID 定义	12
五、软件系统框架	13
六、软件接口调用	14
6.1 设备初始化	14
6.1.1 打开设备	14
6.1.2 更新设备参数	15
6.1.3 初始化设备	15
6.1.4 查询设备状态	16
6.2 图像打印处理	17
6.2.1 启动任务作业	17
6.2.2 载入图像数据	18
6.2.3 打印控制管理	20
6.2.3.1 获取打印状态	20
6.2.3.2 获取 PASS 信息	21
6.2.3.3 执行 PASS 打印	21
6.2.3.4 暂停 PASS 打印	24
6.2.4 停止任务作业	24
6.3 内存打印处理	25
6.3.1 启动任务作业	25
6.3.2 载入打印数据	25
6.3.2.1 启动写数据	25
6.3.2.2 查询写状态	25
6.3.3 打印控制管理	26
6.3.4 停止任务作业	27
6.4 温度电压控制	28
6.4.1 温度电压的设定	28

6.4.2 温度电压的获取.....	29
6.4.3 波形重新加载.....	30
6.5 关闭设备连接.....	31
6.6 驱动卡控制.....	31
6.6.1 设置有效驱动卡.....	31
6.6.2 设置 Fire 信号输出.....	32
6.7 编码位置信息.....	32
6.7.1 获取打印编码.....	32
6.7.2 获取轴编码.....	33
6.8 系统控制.....	33
6.8.1 信号控制.....	33
6.8.1.1 信号状态.....	33
6.8.1.2 零位限制负向.....	34
6.8.1.3 限位使能.....	34
6.8.1.4 轴编码初始化.....	35
6.8.1.5 零位复位计数.....	35
6.8.2 双 Y 轴控制.....	36
6.8.2.1 端口使能.....	36
6.8.2.2 设置同步误差.....	36
6.8.2.3 查询同步误差.....	37
6.8.2.4 查询同步状态.....	37
6.8.3 运动使能.....	38
6.8.4 运动初始化.....	38
6.8.5 运动启动.....	39
6.8.6 运动状态.....	40
6.8.7 运动停止.....	40
6.9 ADIB 控制.....	41
6.9.1 负压和温度设置和回读.....	41
七、系统参数的计算.....	42
7.1 X 运动范围计算.....	42
7.2 Y 步进量计算.....	42
7.3 常用计算公式.....	43
7.3.1 mm 转 dot.....	43
7.3.2 dot 转 mm.....	43
7.3.3 栅距 μ 转 DPI.....	43
7.3.4 打印分频值.....	43
八、喷头校准说明.....	44
8.1 X 向偏差校准图.....	44
8.1.1 喷头内部 X 校准区域.....	44

8.1.2 喷头组间 X 校准区域.....	44
8.1.3 喷头组间 Y 校准区域.....	45
8.1.3 喷头喷嘴状态区域.....	45
8.2 往返差校准图.....	46
8.3 喷嘴状态图.....	47
8.4 喷头机械安装.....	48
九、PHMAN 工具说明	49
9.1 驱动卡通道号.....	49
9.2 喷头位置参数.....	50
十、附录	51
PRTIMG_LAYER,*LPPRTIMG_LAYER	51
PassDataItem,*LPPassDataItem	51
PrtRunInfo,*LPPrtRunInfo	52
PRTJOB_ITEM,*LPPRTJOB_ITEM	52
RYSYS_PARAM,*LPRYSYS_PARAM	53
PRINTER_INFO,*LPPRINTER_INFO	53
DRV_WAVEFORM,*LPDRV_WAVEFORM	54
DRVINFO,*LPDRVINFO	54
ADIB_PARAM,*LPADIB_PARAM	54

一、硬件系统连接

1.1 板卡连接示意图



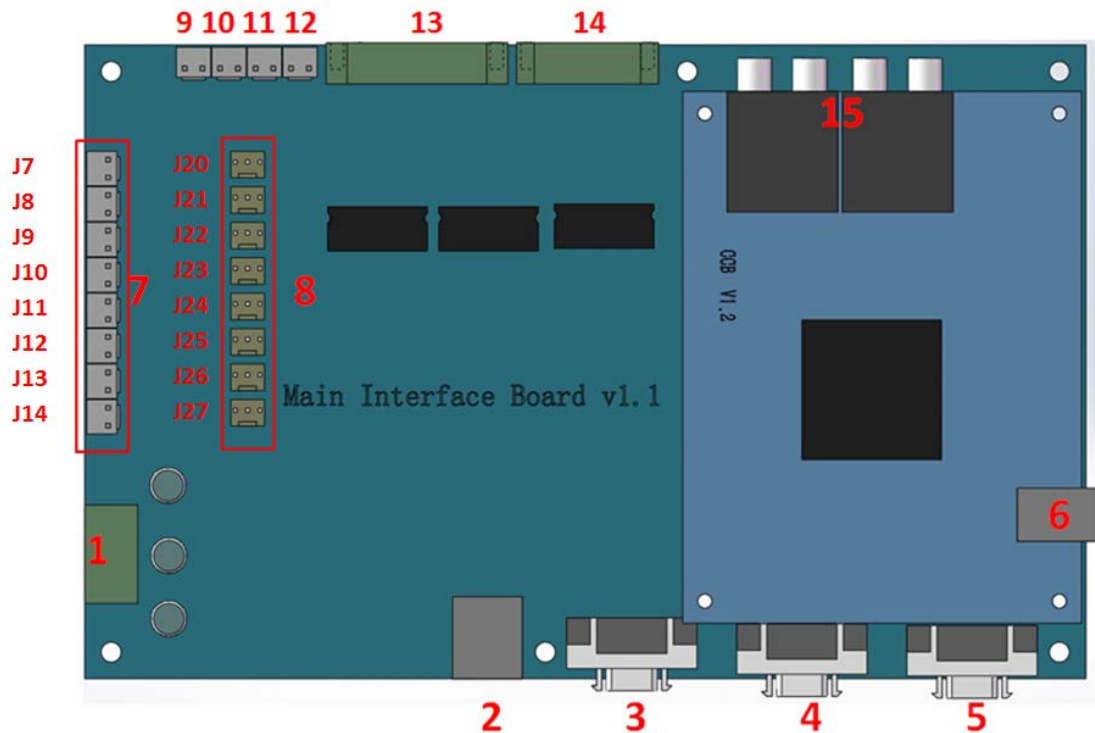
1.1.1 端口说明

板卡端口的简单描述，详见 1.3 电源端口说明

- ① 主板卡电源输入端口
- ② 负压卡网线连接端口
- ③ 车头卡电源输入端口
- ④ 驱动卡电源输入端口

1.2 板卡端口介绍

1.2.1 主板



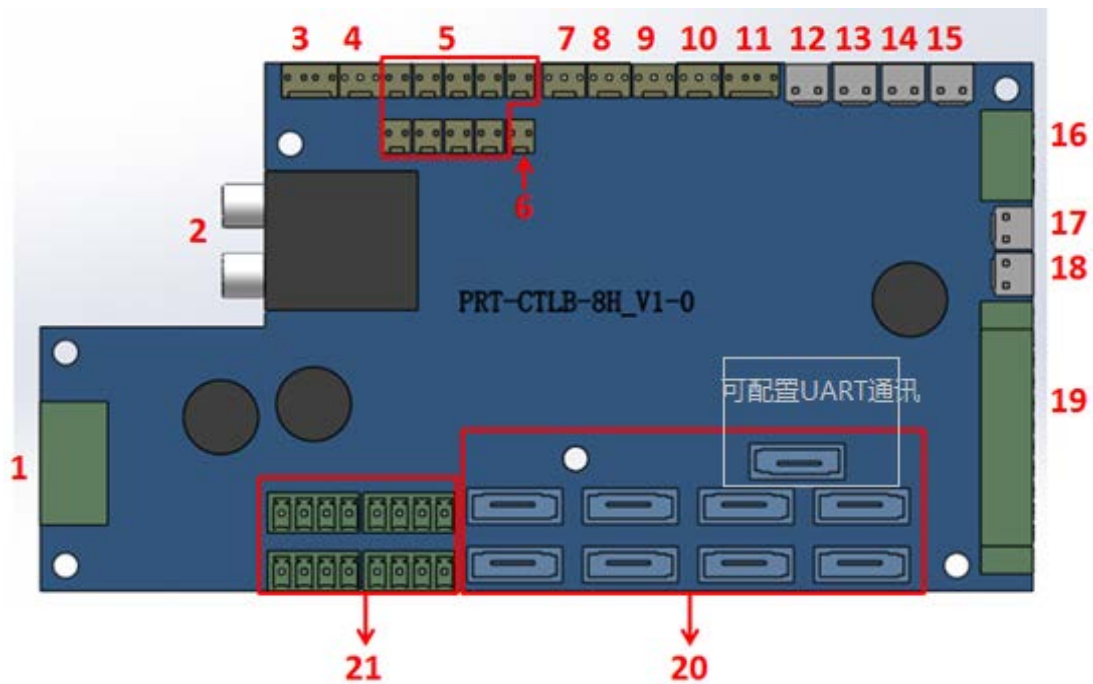
端口介绍：

编号	接口说明
1	24V 输入电源接口
2	标准网络端口（RS485 全双工和半双工）
3	X 电机接口
4	Y1 电机接口
5	Y2 电机接口
6	USB3.0 接口
7	8 路输出接口（见下表）
8	8 路输入接口（见下表）
9	左急停
10	右急停
11	左限位
12	右限位
13-14	墨路接口
15	2 路光纤通讯接口

输入和输出端口功能说明：

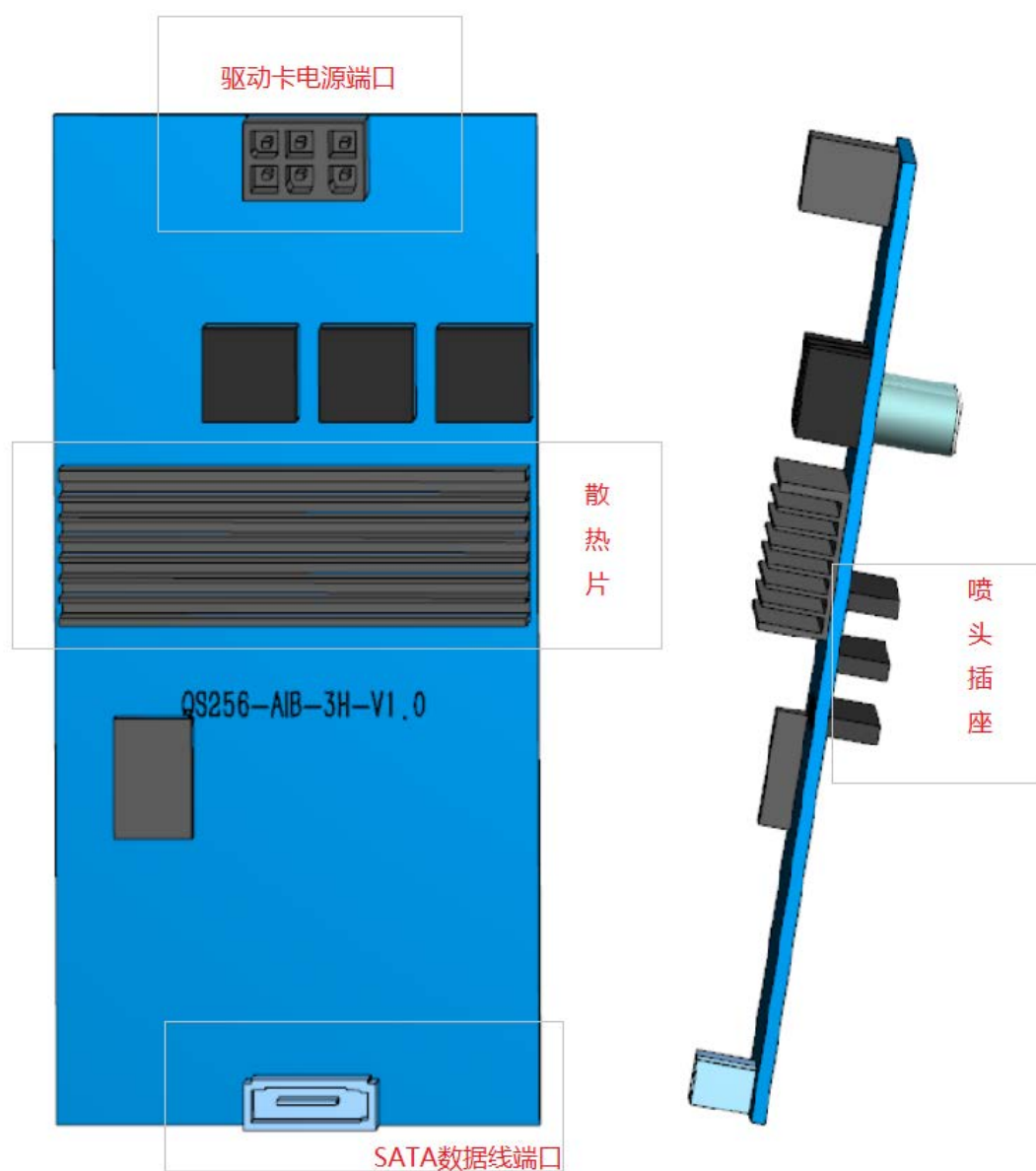
信号形式	接口名称	接口定义	接口定义说明
输入	J20	X 零位	用于 X 轴位置计数复位
	J21	Y 正限位	安装于 Y 轴正向端点的限位信号 (Y1,Y2 轴共用限位)
	J22	Y 负限位	安装于 Y 轴负向端点的限位信号 (Y1,Y2 轴共用限位)
	J23	Y1 零位	用于 Y1 轴位置计数复位
	J24	工作就绪	用于启动打印队列
	J25	Y2 零位	用于 Y2 轴位置计数复位
	J26	保留	保留给其他自定义功能
	J27	保留	保留给其他自定义功能
输出	J7	允许扫描运动启动	运动系统和打印系统独立控制时，为了同步
	J8	打印状态指示	运动系统和打印系统独立控制时，为了同步
	J7-J11	保留	保留给其他自定义功能
	J12	UV 灯功率	控制 UV 灯的功率开关, 打印时如果车头不动达 5 秒以上将自动关闭
	J13	UV 灯快门 1	控制左 UV 灯的快门开关
	J14	UV 灯快门 2	控制右 UV 灯的快门开关

1.2.2 车头板

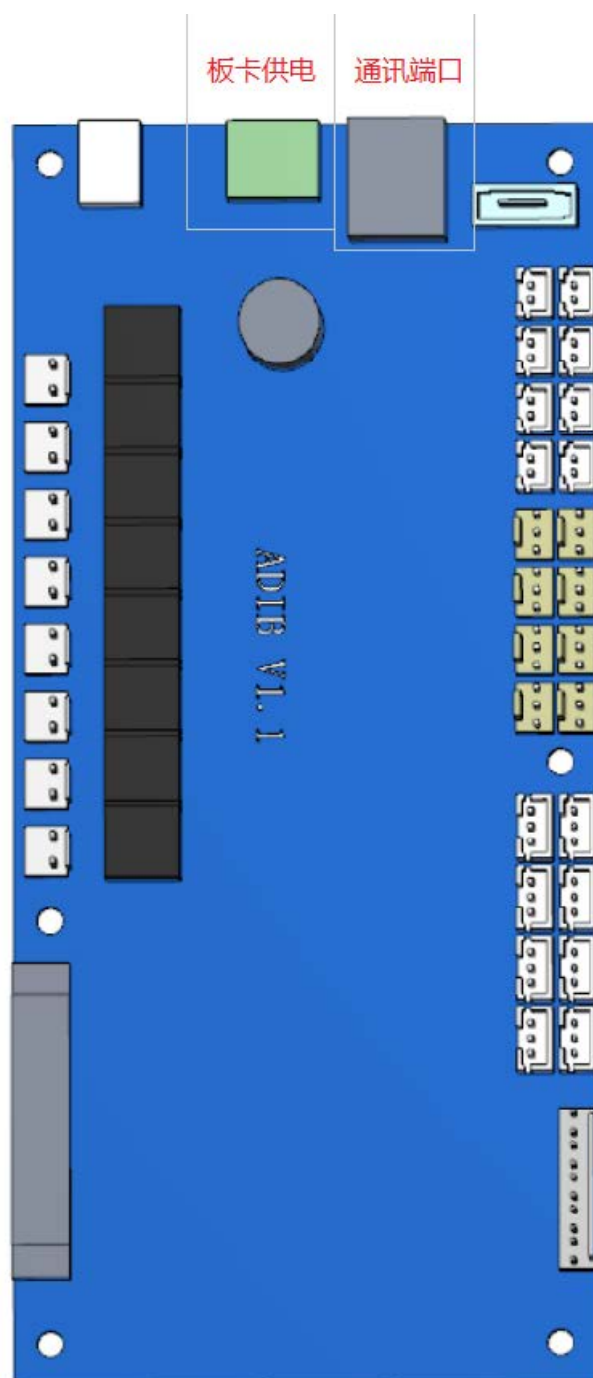


编号	接口说明
1	24V(DC/5A)输入电源接口
2	光纤通讯接口
3	车头 X 向光栅编码器接口
4	寻边传感器接口
5	副墨盒液位信号
6	液位保护墨盒（瓶）接口
7	车头升降零位
8	预留接口
9	车头升降正限位
10	车头升降负限位
11	车头升降光栅编码器接口
12	车头 X 向防撞急停接口
13	预留接口
14	预留接口
15	预留接口
16	车头升降电机驱动器接口
17	车头升降测高感应器输入接口
18	预留输出
19	拓展接口
20	8 路喷头驱动板 SATA 接口+ 1 路可配置 SATA 端口
21	8 路 37V 输出接口

1.2.3 驱动卡



1.2.3 负压卡

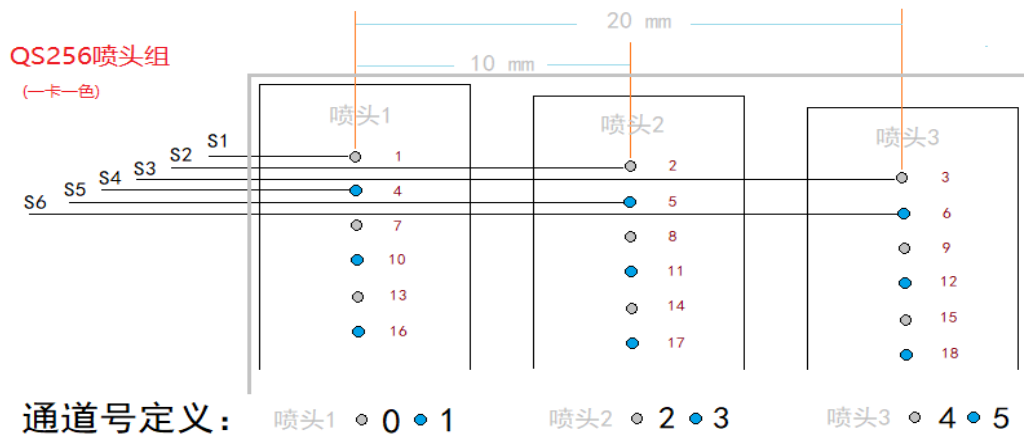


ADIB 负压卡用来控制负压和外部加热，需要用到指定的温度传感器和负压传感器，详见
([ADIB v1.1 使用手册](#))

二、喷头模组定义

2.1 一卡一色

即一个驱动卡对应一种墨水，喷头需要物理上进行Y向错位安装，提高物理Y向DPI



S1	_____	0
S2	_____	2
S3	_____	4
S4	_____	1
S5	_____	3
S6	_____	5

注：喷头1 S1的喷嘴1为该模组起始嘴

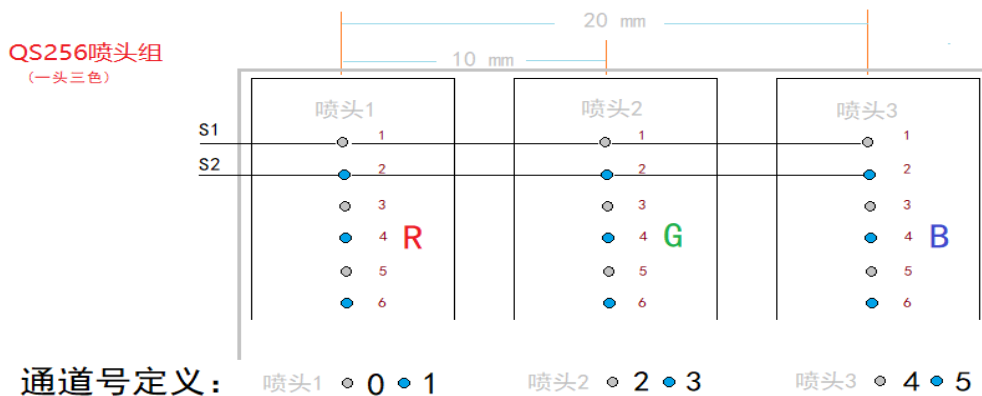
喷头组定义说明:

S1、S2、S3、S4、S5、S6 分别表示喷头组的6列嘴，QS256喷头物理上一列嘴，数据控制上奇偶嘴分两列，所以一个QS256 有两个通道号

图示喷头间距为虚拟，具体请以实际安装为准

2.2 一卡三色

即一个驱动卡对应三种墨水，喷头只需要统一安装方向即可，对Y向无要求



R-S1	_____	0
R-S2	_____	1
G-S1	_____	2
G-S2	_____	3
B-S1	_____	4
B-S2	_____	5

注：S1的喷嘴1为该喷头起始嘴

喷头组定义说明:

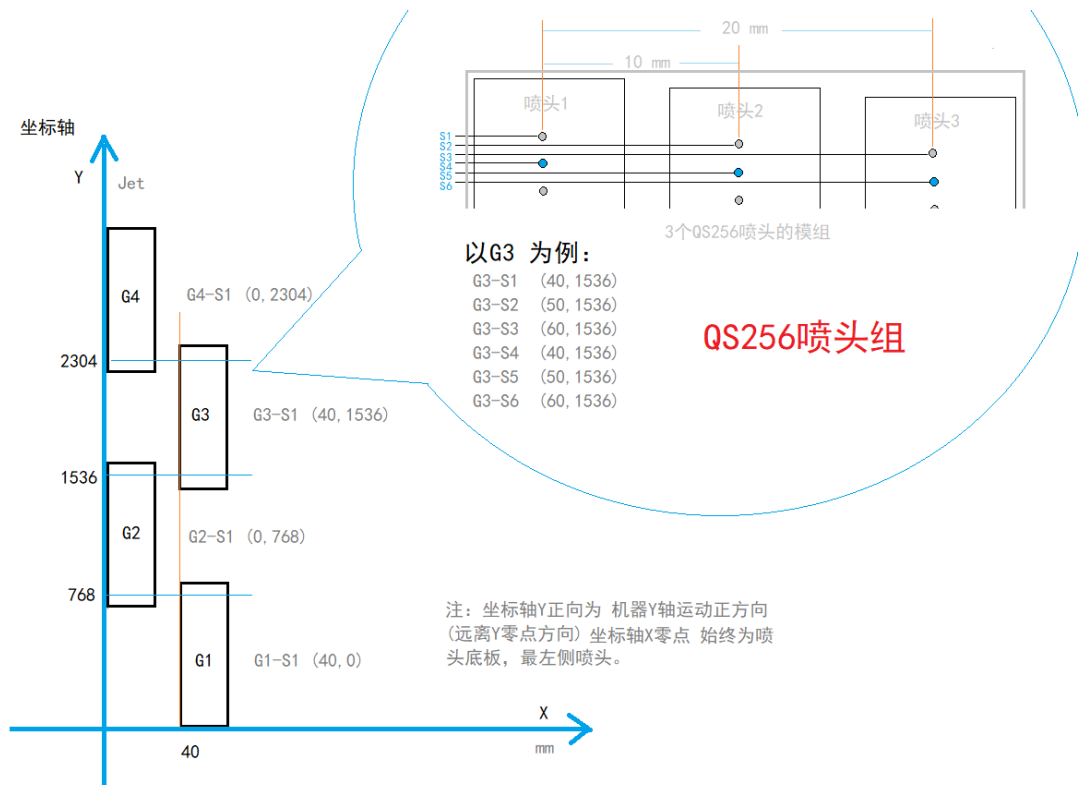
R(0、1)、G(2、3)、B(4、5) 分别表示喷头组的6列嘴，QS256喷头物理上一列嘴，数据控制上奇偶嘴分两列，所以一个QS256 有两个通道号

图示喷头间距为虚拟，具体请以实际安装为准

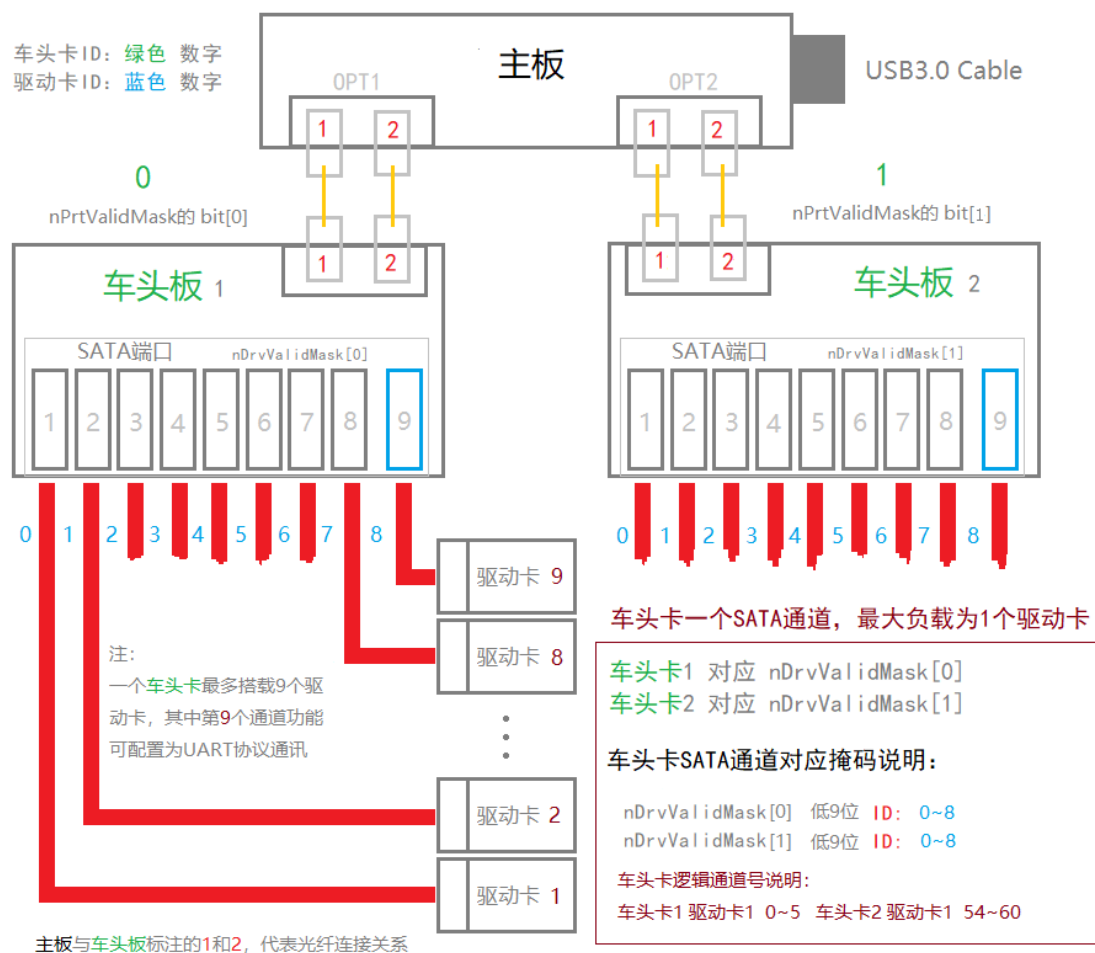
图示的喷嘴号，对应为 PASS 图像的数据行号

三、坐标轴的定义

X 零点始终为喷头底板最左侧喷头, Y 正向为 Y 轴前进的方向, 以此为依据建立坐标轴。
(需要注意的是 X 编码计数往坐标轴 X 正向是需要递增的, 要求与坐标系方向一致, 否则需要通过系统参数来配置。)



四、系统卡 ID 定义

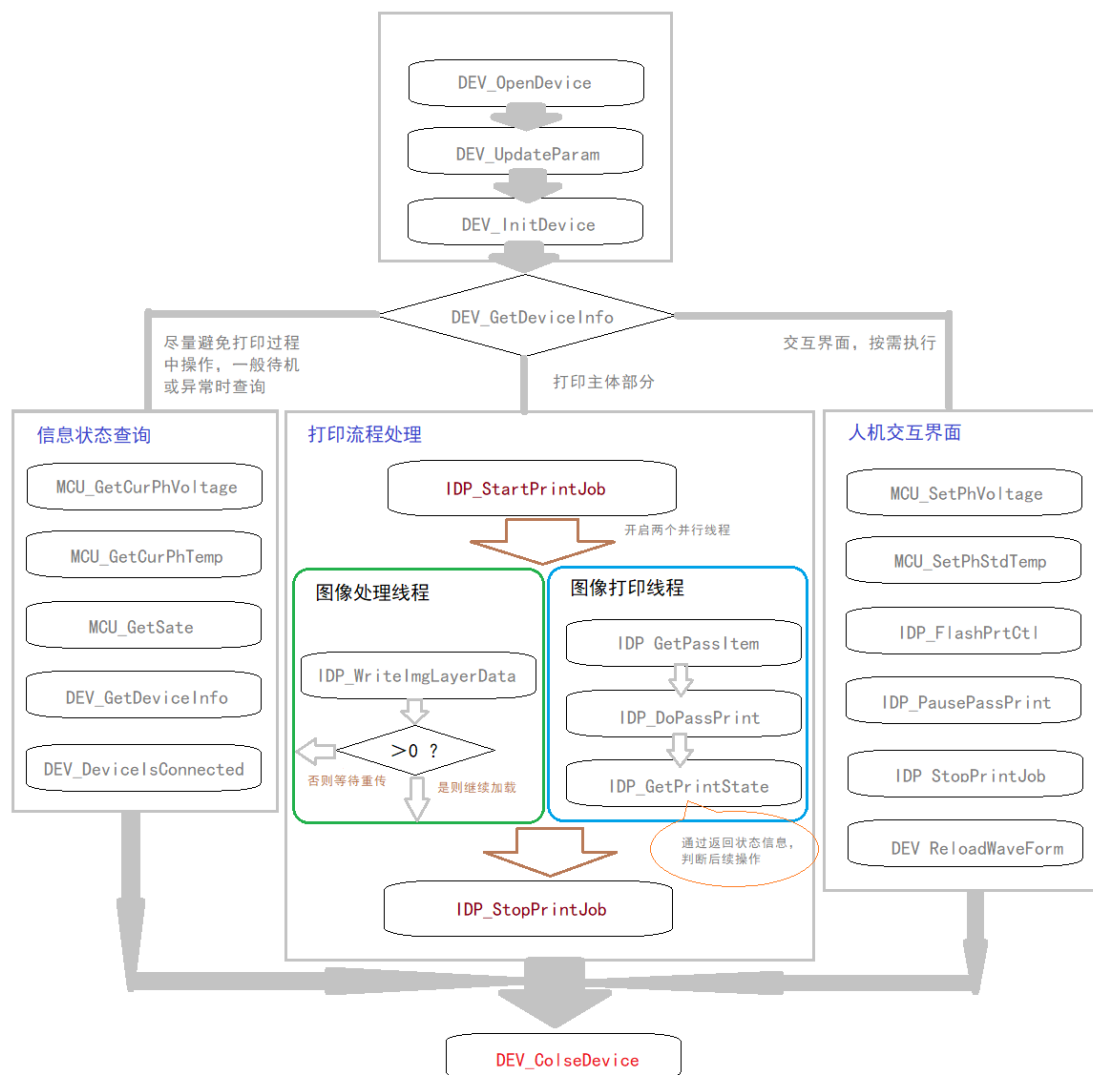


注意: 车头卡 1 和车头卡 2 的第一个 SATA 端口对应的数据通道号是不一样的

驱动卡 ID 和逻辑通道号对应关系如下:

	0	1	2	3	4	5	6	7	8
车头卡 1	0~5	6~11	12~17	18~23	24~29	30~35	36~41	42~47	48~53
车头卡 2	54~59	60~65	66~71	72~77	78~83	84~89	90~95	96~101	102~107

五、软件系统框架



六、软件接口调用

接口调用部分主要介绍 RYPrtCtler.dll 的接口操作说明，主体总体分为三个模块，设备初始化、图像打印处理和温度电压控制，以及最终的关闭设备，图像打印处理和温度电压控制是必须设备初始化完成后才能正常执行的，详细模块介绍如下

6.1 设备初始化

Step1：调用 Dev_OpenDevice 使 PC 端与板卡系统建立通讯

Step2：调用 Dev_UpadteParam 更新设备参数，主要是初始化板卡波形参数

Step3：调用 Dev_InitDevice 初始化设备运行状态，此过程会加载 Step2 中波形参数

Step4：调用 Dev_GetDeviceInfo 查询需要的状态和信息，用来检查初始化设备状态

以上四个步骤完成初始化操作

6.1.1 打开设备

UINT _stdcall Dev_OpenDevice(HANDLE notifyhand, char* szDllFilePath);

参数

notifyhand

指定应用程序实例窗口句柄。

szDllFilePath

指定 fkel.dll 文件路径，路径需要以双斜杠结尾。

返回值

- 0 执行成功
- 1 示例窗口句柄为 NULL
- 2 PCS.pcm 喷头配置文件加载失败
- 3 fkel.dll 加载失败
- 4 板卡授权检测失败

示例

```
UINT nRetVal=Dev_OpenDevice(m_hWnd,"D:\\visual studio 2010\\debug\\");
If(nRetVal>0)
{
    AfxMessageBox(_T("初始化失败!"));
}
//否则，执行成功
```

6.1.2 更新设备参数

BOOL _stdcall DEV_UpdateParam(LPRYSYS_PARAM pParam);**参数**

pParam

指向参数结构体 RYSYS_PARAM 的指针,结构体详见附录。

返回值

TRUE 执行成功

FALSE 执行失败,一般只有指向地址不可访问才会失败。

示例

```
RYSYS_PARAM sysParam;  
sysParam.nParamVer=0x21080809;  
sysParam.nParamSzie=sizeof(RYSYS_PARAM);  
...  
//参数结构体的全部参数均由上位机开发人员赋值,喷绘扫描行业,由于波形  
//和偏差关联性较大,开发者一般会定//义为参数文件加载形式,多份参数自  
//由切换  
Dev_UpadteParam(&sysParam);  
//会在指定的 log 目录下 waveform 里形成波形预视图
```

6.1.3 初始化设备

UINT _stdcall DEV_InitDevice(UINT nXEnclnitVal);**参数**

nXEnclnitVal

指定的初始编码值,为运动系统车头扫描方向的当前编码值

返回值

0 执行成功

-1 初始化未检测到车头卡连接,需要检查光纤线连接

-2 检测车头卡 1 连接时,未检测到该车头卡上有驱动卡连接

-3 检测车头卡 2 连接时,未检测到该车头卡上有驱动卡连接

-10 车头卡 1 负载驱动卡存在波形加载失败,详见 nDrvWaveErrMask[0]

-11 车头卡 2 负载驱动卡存在波形加载失败,详见 nDrvWaveErrMask[1]

示例

```
UINT nSysInitEncVal=0x10000; //此值来自运动系统当前 X 编码  
// 距离 S mm 转换为 Encode,DPI 为扫描方向光栅 DPI  
// Encode = ((S/25.4)*DPI)  
UINT nRetVal=Dev_InitDevice(nSysInitEncVa);  
If(nRetVal<0)  
{  
    AfxMessageBox(_T("初始化失败!"));//  
}  
//否则,执行成功
```

6.1.4 查询设备状态

LPPRINTER_INFO _stdcall DEV_GetDeviceInfo();

参数

void

返回值

指向 PRINTER_INFO 结构的指针，结构体详见附录。

示例

```
LPPRINTER_INFO pSysInfo=Dev_GetDeviceInfo();
/////判断设备是否为 USB3.0 驱动
pSysInfo->bSuperDevice

/////获取车头卡连接掩码
pSysInfo->nPrtValidMask //bit[0] 车头卡 1, bit[1] 车头卡 2
/////获取车头卡连接的驱动卡掩码
pSysInfo->nDrvValidMask[0] //车头卡 1
pSysInfo->nDrvValidMask[2] //车头卡 2
/////获取主板卡版本信息
pSysInfo->nMainFpgaVer
/////获取车头卡版本信息
pSysInfo->nCarFpgaVer[0]
pSysInfo->nCarFpgaVer[1]
/////获取驱动卡版本信息
for(int p=0;p<2;p++) //车头卡 ID 最大 2 个车头卡
{
    for(int d=0;d<32;d++) //一个车头卡最多搭载 32 个驱动卡
    {
        //驱动卡 MCU 程序版本
        nFMVer[p][d]=pSysInfo->sysDrvInfo[p*32+d].nFMVersion;
        //驱动卡 MCU 运行状态
        nState[p][d]=pSysInfo->sysDrvInfo[p*32+d].nPtwarnState;
        //驱动卡 FPGA 程序版本
        nFpgaVer[p][d]=pSysInfo->sysDrvInfo[p*32+d].nFpgaVersion;
    }
}
```

bool _stdcall DEV_DevicelsConnected();

参数

void

返回值

TRUE	设备已连接
FALSE	设备断开连接

示例

```
if(!Dev_DevicelsConnected())
    AfxMessageBox(_T("设备已断开连接")); //可执行重新初始化
//该状态可实时监控
```

6.2 图像打印处理

6.2.1 启动任务作业

int _stdcall IDP_StartPrintJob(LPPRTJOB_ITEM pJobItem);

参数

pJobItem

指向作业结构 PRTJOB_ITEM 的指针，传递作业处理参数

返回值

0	执行成功
-100001	Y 偏差过大，相邻组分离
-100002	有效打印喷嘴数小于 1
-200000	上次任务没有结束
-200001	打印管理线程启动失败
-200010	车头卡 1 打印参数设置失败
-200011	车头卡 2 打印参数设置失败

示例

```
/******
启动和关闭作业任务，每个沙箱作业，都需要执行一次
需要先启动 Job，然后开启载图和打印线程，作业完成后
需要关闭 Job，用来完全释放内存，详见 IDP_StopPrintJob
*****/
PRTJOB_ITEM _JobItem; //打印作业对象
ZeroMemory(&_JobItem,sizeof(PRTJOB_ITEM));
_JobItem.nJobID=0;
_JobItem.nPixelGrayBits=1;
_JobItem.nPrtXEncPos=0x100018; //X 打印起始位置编码值
if(IDP_SartPrintJob(&_JobItem)<0)
{
    AfxMessageBox(_T("Can't Print"));
}
else
{
    AfxBeginThread(ProcImgThread,this);//上位机开启图像处理线程
    AfxBeginThread(ProcPrtThread,this);//上位机开启打印管理线程
}
//执行成功，可开启两个并行线程，分别处理图像数据，和打印管理
```

6.2.2 载入图像数据

```
int __stdcall IDP_WriteImgLayerData(LPRTIMG_LAYER lpLayerInfo,LPBYTE  
pSrcBuf[],int nBytes);
```

参数

lpLayerInfo

指向指定图层的信息结构

pSrcBuf[]

指向图层的数据区域，多个数据通道，注意不包括信息结构

nBytes

指定图层数据区的字节数，多通道时，需要该参数一致

返回值

- >0 指定图层打印执行时的 PASS 总数
- 110000 图层添加的序号不满足增量关系
- 110001 PC 内存不足
- 110002 PASS 计算小于 0

示例

```
//示例以单色位图为例  
CFile file;  
int nColors=3; //R G B 为例  
if(!file.Open(_T("D:\\test.bmp"),CFile::modeRead))  
{  
    AfxMessageBox(_T("Failed to open file."));  
    return;  
}  
int nPrtDataSize=file.GetLength();  
LPBYTE pDataBuf=new BYTE[m_nPrtDataSize+1024];  
ZeroMemory(pDataBuf, nPrtDataSize);  
file.Read(pDataBuf,nPrtDataSize);  
file.Close();  
BITMAPFILEHEADER* pfheader;  
BITMAPINFOHEADER* pbmpheader;  
pfheader=(BITMAPFILEHEADER*) pDataBuf;  
pbmpheader=(BITMAPINFOHEADER*)( pDataBuf +sizeof(BITMAPFILEHEADER));  
if(pbmpheader->biBitCount!=1) //仅支持单点打印  
{  
    AfxMessageBox(_T("不支持非单点图像打印"));  
    delete pDataBuf;  
    return;  
}  
int nVIDOffset=pfheader->bfOffBits; nPrtDataSize-= nVIDOffset;
```

```
RGBQUAD* pcolor = (RGBQUAD*)( pDataBuf +sizeof(BITMAPFILEHEADER)
+sizeof(BITMAPINFOHEADER));
COLORREF clrZeroColor = RGB(pcolor[0].rgbRed,pcolor[0].rgbGreen,
pcolor[0].rgbBlue);
for(int c=0;c< nColors;c++) //颜色数 3 个
{
    CopyMemory(m_pPrtData[c], pDataBuf+ nVIDOffset, nPrtDataSize); //图层
}
if(clrZeroColor==0)
{
    for(int c=0;c< nColors;c++) //颜色数 3 个
    {
        for(int s=0;s<pbmpheader->biSizelImage;s++)
            m_pPrtData[c] [s]=~ m_pPrtData[c] [s];
    }
}
////////处理图层信息
PRTIMG_LAYER _LayerInfo; //打印图层对象
ZeroMemory(&_LayerInfo,sizeof(PRTIMG_LAYER));
_LayerInfo.nXDPI=317; //图像 XIDPI
_LayerInfo.nYDPI=300; //图像 YIDPI 计算时暂时不用, 默认为喷头组 DPI
_LayerInfo.nBytesPerLine=pbmpheader->biSizelImage/pbmpheader->biHeight;
//每行数据字节数
_LayerInfo.nWidth=pbmpheader->biWidth;
_LayerInfo.nHeight=pbmpheader->biHeight;
_LayerInfo.nLayerIndex=(g_nLayerCount);
_LayerInfo.nColorCnts= nColors; //颜色个数
_LayerInfo.nPrtDir=1;
_LayerInfo.nPrtFlag=1; //双向打印 bit[0] 控制单双向打印
//////////
int nRet=-1;
do
{
    nRet=IDP_WriteImgLayerData(&_LayerInfo, m_pPrtData, nPrtDataSize);
    if(nRet>0)
        break;
    else
    {
        //分析错误号, 做出相应处理
    }
}while( nRet<=0);
载入成功后则可以继续执行以上流程, 添加下一个图层数据
```

6.2.3 打印控制管理

打印管理有三个步骤：

- Step1：调用 IDP_GetPassItem 获取 PASS 信息体，用于提供给运动系统的控制
- Step2：调用 IDP_DoPassPrint 执行指定 PASS 打印，用于指定 PASS 的打印使能
- Step3：调用 IDP_GetPrintState 获取当前打印状态，用于判断打印状态和后续操作

以上三个步骤即可操作完成打印，系统也提供 IDP_DoPassPrint2 接口，用来图层托管打印处理，同时提供了两个人机交互接口 IDP_PausePassPrint 和 IDP_StopPrintJob，用来暂停当前 PASS 和停止打印任务，详见 6.2.3.4 暂停和停止打印

6.2.3.1 获取打印状态

bool __stdcall IDP_GetPrintState(LPPrtRunInfo pRTinfo);

参数

pRTinfo
指向需要缓存 PRTRunInfo 的结构体指针

返回值

TRUE 执行成功
FALSE 执行失败

示例

```
PrtRunInfo RTinfo;  
IDP_GetPrintState(&RTinfo);  
RTinfo.bJobPrtRuning; //打印作业运行状态  
if(1==RTinfo.nPrtState)  
{  
    AfxMessageBox(_T("当前 PASS 打印进行中"))  
}  
else if(3==RTinfo.nPrtState)  
{  
    AfxMessageBox(_T("当前系统正在闪喷"))  
}  
else if(0==RTinfo.nPrtState)  
{  
    AfxMessageBox(_T("系统空闲"));  
}
```

6.2.3.2 获取 PASS 信息

LPPassDataItem `_stdcall IDP_GetPassItem(unsigned int nLayerIndex,int nPassID);`

参数

nLayerIndex

指定图层的索引号，与 IDP_WriteImgLayerData 中 LPPRTIMG_LAYER 结构中相对应

nPassID

指定 PASS 的索引号，索引号 0 为第一 PASS

返回值

指向指定图层指定 PassDataItem 的结构体指针，详见附录

NULL 获取失败

非空 执行成功

示例

```
LPPassDataItem pPrtPassDes;
int nLayerIndex=0;    //上位机已经添加过的图层索引号
int nPassCount=4;     //IDP_WriteImgLayerData 的返回值
for(int i=0;i< nPassCount;i++)
{
    pPrtPassDes= IDP_GetPassItem(nLayerIndex,i);
    if(pPrtPassDes)
    {
        IDP_DoPassPrint(pPrtPassDes);    //执行 Pass 打印
    }
}
//获取指定 PASS 信息，并执行打印
```

6.2.3.3 执行 PASS 打印

该部分介绍两个执行打印的方法 IDP_DoPassPrint 和 IDP_DoPassPrint2，前者只能执行指定 PASS 的打印，后者不仅可以指定 PASS 打印，还可托管单个图层打印，执行 PASS 打印前，需要对打印控制状态和 PASSITEM 信息进行判断，打印控制器是否处于空闲状态，以及 PASS 数据是否准备就绪，在接口示例中有详细操作，详见如下：

bool __stdcall IDP_DoPassPrint(LPPassDataItem pPassItem);**参数**

pPassItem

执行需要打印的PassDataItem的指针，由IDP_GetPassItem获取

返回值

TRUE 执行成功

FALSE 执行失败

示例

```
LPPassDataItem pPrtPassDes;
int nLayerIndex=0;    //上位机已经添加过的图层索引号
int nPassCount=4;    //IDP_WriteImgLayerData 的返回值
for(int i=0;i< nPassCount;i++)
{
    pPrtPassDes= IDP_GetPassItem(nLayerIndex,i);
    if(pPrtPassDes)
    {
        if(!IDP_DoPassPrint(pPrtPassDes))
        {
            //查看打印状态，作相应处理
        }
    }
    //计算当前 PASS 运动系统需要的参数
    int nYLastJetOff=pPrtPassDes. nMinJet0ImgLinePos; //上 PASS 的行偏移数
    int nYCurJetOff=pPrtPassDes. nMinJet0ImgLinePos; //当前 PASS 的行偏移
    int nXStartPos=pPrtPassDes. nStartEncPos //当前 PASS 打印起始编码值
    int nXValidCol=pPrtPassDes. nValidPrtCols;    //当前 PASS 数据有效列数
    int nPrtcess=pPrtPassDes. nPrtPrecession ;    //光栅打印分频值
    int nXEndPos=0;                                //X 打印结束编码值
    if(pPrtPassDes.bPrtDir)
    {
        nXEndPos= nXStartPos + nPrtcess* nXValidCol;
    }
    else
    {
        nXEndPos= nXStartPos - nPrtcess* nXValidCol;
    }
} //X 车头运动的范围，图像区域的起点 nXStartPos，至终点 nXEndPos，两边
需要加上缓冲区
int nYDPI =300; //图层 Y 向 DPI，当前系统定义的打印方式，必须等于喷头 DPI
float fYmmStepSize=( (nYCurJetOff- nYLastJetOff)/ nYDPI)*25.4;
//Y 轴运动需要运动的距离 mm
(即 float fYmmStepSize =(▲nMinJet0ImgLinePos/nYDPI)*25.4 mm
▲表示差值)
```

```
bool __stdcall IDP_DoPassPrint2(unsigned int nLayerIndex,int nPassID);
```

参数

nLayerIndex

指定图层的索引号，与 IDP_WriteImgLayerData 中 LPPRTIMG_LAYER 结构中相对应

nPassID

指定PASS的索引号，索引号0为第一PASS，当传入-1时，可托管该图层打印

返回值

TRUE 执行成功

FALSE 执行失败

示例

方法 1:

```
int nLayerIndex=0; //上位机已经添加过的图层索引号
int nPassCount=4;   //IDP_WriteImgLayerData 的返回值
LPPassDataItem pPrtPassDes;
for(int i=0;i< nPassCount;i++)
{
    if(!IDP_DoPassPrint2(nLayerIndex,i)) //单 PASS 打印执行
    {
        //查看打印状态，作相应处理
    }
    else
    {
        pPrtPassDes= IDP_GetPassItem(nLayerIndex,i);
        if(pPrtPassDes)
        {
            //计算当前 PASS 运动系统需要的参数
        }
    }
}
```

方法 2:

```
int nLayerIndex=0; //上位机已经添加过的图层索引号
int nPassCount=4;   //IDP_WriteImgLayerData 的返回值
LPPassDataItem pPrtPassDes[4];
for(int i=0;i< nPassCount;i++)
{
    pPrtPassDes[i]= IDP_GetPassItem(nLayerIndex,i);
    if(!pPrtPassDes[i])
        AfxMessageBox(_T("当前 PASS 信息获取失败"));
} //每 PASS 运动参数形成队列传递给运动系统
if(!IDP_DoPassPrint2(nLayerIndex,-1)) //托管该图层打印
    AfxMessageBox(_T("打印启动失败!"));
```

6.2.3.4 暂停 PASS 打印

bool __stdcall IDP_PausePassPrint(bool bPause);**参数**

bPause

等于 1 时，会暂停当前 PASS 打印，等于 0 时，可继续打印，该函数前提为设备系统此过程中未断过电

返回值

TRUE 执行成功

FALSE 执行失败

示例

```
//人机交互界面处理
Void CDialog::onBtnCheck()
{
    CButton* pBtn = (CButton*)GetDlgItem(IDC_CHECK)
    if(pBtn->GetCheck())
        IDP_PausePassPrint(FALSE);
    else
        IDP_PausePassPrint(TRUE); //设备就绪时，执行
}
//异常情况干扰或操作者执行，由开发者灵活应用
```

6.2.4 停止任务作业

bool __stdcall IDP_StopPrintJob();**参数**

void

该函数需要与 IDP_StartPrintJob 成对使用，用来释放其分配的内存空间

返回值

TRUE 执行成功

FALSE 执行失败

示例

```
//人机交互界面和打印流程处理
Void CDialog::onStopJob()
{
    IDP_StopPrintJob();
}
//异常情况干扰或操作者执行，由上位机开发者灵活应用
注意：需要与 IDP_StartPrintJob 成对使用
```

6.3 内存打印处理

6.3.1 启动任务作业

调用 IDP_StartPrintJob，详见图像打印处理下 6.2.1

6.3.2 载入打印数据

6.3.2.1 启动写数据

bool __stdcall DDR_WritelmgData(LPBYTE pdata, int nBytes, UINT

nMemAdr);

参数

pdata

数据缓存区的指针

nBytes

数据缓存区的大小

nMemAdr

指定写入板卡内存的物理地址，与打印的地址匹配

返回值

TRUE 执行成功

FALSE 执行失败

示例

```
//人机交互界面和打印流程处理
LPBYTE pSrcData= new BYTE[0x2000000]; //单次写入最大 64 mb
BOOL bStartWD=DDR_WritelmgData(pSrcData,(0x2000000+0x10000),0);
if(!bStartWD)
    AfxMessageBox(_T("写数据启动失败!"));
//该接口为异步接口，启动成功，不代表数据已经完全写入，需要判断
注意：需要与 DDR_WritelsOver 成对使用，用来判断当前写操作是否已经完成
```

6.3.2.2 查询写状态

bool __stdcall DDR_WritelsOver();

参数

void

返回值

TRUE 当前写操作已经完成

FALSE 当前写操作正在进行

示例

```
//先执行 DDR_WritelmgData 写操作，后 DDR_WritelsOver 判断执行情况
LPBYTE pSrcData= new BYTE[0x2000000]; //单次写入最大 64 mb
```

```
if(!DDR_WriteImgData(pSrcData,(0x2000000+0x10000),0))
{
    AfxMessageBox(_T("写数据启动失败! "));
    return ;
}
DWORD dwTick=GetTickCount();
while(!DDR_WritelsOver())
{
    if(GetTickCount()-dwTick>20000)
    {
        AfxMessageBox(_T("写数据超时! "));
        return ;
    }
}
//写入成功
注意: 需要与 DDR_WriteImgData 启动成功后再调用才有意义
```

6.3.3 打印控制管理

int **__stdcall** **DDR_StartPrint**(**UINT** nXPrecision,**UINT** nPdMemAdr[],**UINT** nPixelWidth,**UINT** nStartEncPos,**bool** bPrtDir);

参数

nXPrecision

光栅点火间隔

nPdMemAdr[]

指定起始打印的板卡内存的物理地址,多个控制器

nPixelWidth

打印的扫描方向的像素宽度

nStartEncPos

指定打印的起始打印位置编码

bPrtDir

指定打印方向, 1 计数增大方向 0 计数减小方向

返回值

-220001 打印精度设置失败

-220001 打印精度设置失败

>0 启动成功

示例

```
//假设系统光栅 DPI 是 0.1um, 4 倍频即 254000 DPI
//假设需要打印的 X 方向 DPI 为 254
UINT nPrecision =254000/254=1000;
```

```
//各打印控制器地址需要在数据处理时计算好，由控制器的喷头数决定
//目前可能需要用到两个控制器（车头卡），假设控制器1接入3个卡，控制器
//2接入3卡，两个控制器的内存数据是可以独立存放的
UINT nPdMemAdr[2]={0, 0x1000000};
UINT nPixelWidth = 10000;//254DPI的情况下1000mm的幅宽
//假设0x100000是运动零点的计数值，打印起点距离零点100mm，那么计数是
//1000 (0x3e8),
UINT nStartEncPos=0x1003e8; //是运动系统初始化后的板卡端计数值，
bool bPrtDir=TRUE; //表示打印方向，是增计数打印还是减计数打印

//参数计算好后，调用接口
if(DDR_StartPrint(nPrecision, nPdMemAdr, nPixelWidth, nStartEncPos, bPrtDir)>0)
{
    ;//开始X方向运动
}
```

6.3.4 停止任务作业

调用 IDP_StopPrintJob，详见图像打印处理下 6.2.4

用来释放内存和初始化操作

6.4 温度电压控制

6.4.1 温度电压的设定

bool __stdcall MCU_SetPhVoltage(float* fstdVcom,unsigned int

nCMID,unsigned int nDrvID);

参数

fstdVcom

执行设定电压的指针，QS256 驱动卡可调电压为 3 个

nCMID

车头卡 ID，0 代表 车头卡 1，1 代表车头卡 2，当前主板 0~1.

nDrvID

驱动卡 ID，0 代表驱动卡 1，范围 0~31.

返回值

TRUE 设定成功

FALSE 设定失败

示例

```
float fstdVoltage[3];
for(int i=0;i<3;i++)
    fstdVoltage[i]=85.0f;
MCU_SetPhVoltage(fstdVoltage,0,0); //设定车头卡 1，驱动卡 1 的电压
MCU_SetPhVoltage(fstdVoltage,1,3); //设定车头卡 2，驱动卡 4 的电压
```

bool __stdcall MCU_SetStdTemp(float* fstdTmp,unsigned int nCMID,unsigned

int nDrvID);

参数

fstdTmp

执行设定温度的指针，QS256 驱动卡可控温度为 1 个，默认 70℃

nCMID

车头卡 ID，0 代表 车头卡 1，1 代表车头卡 2，当前主板 0~1.

nDrvID

驱动卡 ID，0 代表驱动卡 1，范围 0~31.

返回值

TRUE 设定成功

FALSE 设定失败

示例

```
float fProTemp=70.0f;
MCU_SetStdTemp(&fProTemp,0,0); //设定车头卡 1，驱动卡 1 的保护温度
MCU_SetStdTemp(&fProTemp,1,3); //设定车头卡 2，驱动卡 4 的保护温度
```

6.4.2 温度电压的获取

bool __stdcall MCU_GetCurPhVoltage(float* fVoltage,unsigned int
nCMID,unsigned int nDrvID);

参数

fVoltage

指向 4 个电压缓存的指针，QS256 驱动卡前 3 个高压，后 1 个 12v 电压

nCMID

车头卡 ID，0 代表 车头卡 1，1 代表车头卡 2，当前主板 0~1.

nDrvID

驱动卡 ID，0 代表驱动卡 1，范围 0~31.

返回值

TRUE 执行成功

FALSE 执行失败

示例

```
float fVoltage[4];  
MCU_GetCurPhVoltage(fVoltage,0,0); //车头卡 1 驱动卡 1 当前温度值  
fVoltage[0]    //模组内第一路高压  
fVoltage[1]    //模组内第二路高压  
fVoltage[2]    //模组内第三路高压  
fVoltage[3]    //板卡 12v 电压值
```

bool __stdcall MCU_GetCurPhTemp(float* fCurTemp,unsigned int
nCMID,unsigned int nDrvID);

参数

fCurTemp

指向 4 个温度缓存的指针，QS256 驱动卡前 3 个喷头温度，后 1 个板卡温度

nCMID

车头卡 ID，0 代表 车头卡 1，1 代表车头卡 2，当前主板 0~1.

nDrvID

驱动卡 ID，0 代表驱动卡 1，范围 0~31.

返回值

TRUE 执行成功

FALSE 执行失败

示例

```
float fCurTemp[4];  
MCU_GetCurPhTemp(fCurTemp,0,0); //车头卡 1，驱动卡 1 的当前温度  
fCurTemp[0]    //模组内第一路温度  
fCurTemp[1]    //模组内第二路温度  
fCurTemp[2]    //模组内第三路温度  
fCurTemp[3]    //板卡当前温度值
```

bool **__stdcall** MCU_GetSate(**unsigned short*** nState,**unsigned int** nCMID,**unsigned int** nDrvID);

参数

nState

指向缓存的指针，回读 MCU 运行状态值

nCMID

车头卡 ID，0 代表 车头卡 1，1 代表车头卡 2，当前主板 0~1.

nDrvID

驱动卡 ID，0 代表驱动卡 1，范围 0~31.

返回值

TRUE 执行成功

FALSE 执行失败

示例

```
USHORT nState=0xFFFF;    //nState 状态正常值为 0x700，所以故意赋值
MCU_GetSate(&nState,0,0);
//0x701 板卡温度过高
//0x702 板卡电压过载
```

6.4.3 波形重新加载

int **__stdcall** DEV_ReloadWaveForm();

参数

void

该函数用来加载参数结构RYSYS_PARAM中的drvWaveForm[MAX_DRV_CNT]

返回值

0 执行成功

1 车头卡 1 负载驱动卡存在波形加载失败，详见 nDrvWaveErrMask[0]

2 车头卡 2 负载驱动卡存在波形加载失败，详见 nDrvWaveErrMask[1]

示例

```
//当需要关闭上位机软件执行
void CDialog::OnLoadWaveForm()
{
    int nRet=DEV_ReloadWaveForm();
    if(nRet>0)
        AfxMessageBox(_T("波形加载失败! "));    //查询 Dev_GetDeviceInfo
}
LPPRINTER_INFO pSysInfo=Dev_GetDeviceInfo();
pSysInfo->nDrvWaveErrMask[0]; //车头卡 1 驱动卡波形加载失败掩码
```

6.5 关闭设备连接

bool __stdcall DEV_CloseDevice();

参数

void

该函数用来安全退出板卡系统，执行板卡安全掉电等操作

返回值

TRUE 执行成功

FALSE 执行失败

示例

```
//当需要关闭上位机软件执行
Void CDialog::OnDesory()
{
    DEV_CloseDevice();
    CDialog::OnDestory();
}
```

6.6 驱动卡控制

6.6.1 设置有效驱动卡

bool __stdcall DEV_SetValidDrvMask(unsigned int nCMID,unsigned int

nDrvMask);

参数

nCMID

车头卡 ID，0 代表 车头卡 1，1 代表车头卡 2，当前主板 0~1

nDrvMask

初始值从 Dev_GetDeviceInfo()中获取

返回值

TRUE 执行成功

FALSE 执行失败

示例

```
LPPRINTER_INFO pSysInfo=Dev_GetDeviceInfo();
UINT nDrvMask= pSysInfo-> nDrvValidMask[0]; //车头卡 1 为例
Dev_SetValidDrvMask(0, nDrvMask); //正常打印时设置的值
Dev_SetValidDrvMask(0,( nDrvMask&~0x1)); //关闭第一个驱动卡数据
```

6.6.2 设置 Fire 信号输出

bool **__stdcall** DEV_EnableDrvOutput(**unsigned int** nCMID,**unsigned int** nDrvID,**bool** bEnable);

参数

nCMID

车头卡 ID, 0 代表 车头卡 1, 1 代表车头卡 2, 当前主板 0~1.

nDrvID

驱动卡 ID, 0 代表驱动卡 1, 范围 0~31.

bEnable

等于 true 时, 点火信号输出, false 时, 则关闭

返回值

TRUE 执行成功

FALSE 执行失败

示例

```
//以车头卡 1 驱动卡 1 为例
Dev_EnableDrvOutput(0,0,true); //驱动卡端输出墨滴观测系统需要的信号

Dev_EnableDrvOutput(0,0,false); //驱动卡端关闭墨滴观测系统需要的信号
```

6.7 编码位置信息

6.7.1 获取打印编码

unsigned int **__stdcall** DEV_GetPrintEncoderValue();

参数

void

返回值

返回系统实时打印位置编码值。

示例

```
//用来与运动系统的位置进行对比, 检验编码系统的准确性, 作 log 记录

UINT nCurPos=Dev_GetPrintEncoderValue(); //当前车头编码位置
```

6.7.2 获取轴编码

unsigned int __stdcall DEM_GetAxisPosVal(unsigned int nAxis);**参数**

nAxis

指定轴索引号, 0:X轴 1:Y 轴 2:Z 轴

返回值

unsigned int

轴编码器值

示例

```
//用来与运动系统的位置进行对比，检验编码系统的准确性，作 log 记录
```

```
UINT nCurPos= DEM_GetAxisPosVal (0); //当前 X 轴编码位置
```

6.8 系统控制

6.8.1 信号控制

6.8.1.1 信号状态

unsigned int __stdcall DEM_GetAxisLmtZeroState(unsigned int nAxis)**参数**

nAxis

轴索引号

返回值

unsigned int

返回 指定轴 正限位、负限位和零位等信号状态

bit[0] 零位

bit[0] 正限位

bit[0] 负限位

示例

```
//用来与运动系统的位置进行对比，检验编码系统的准确性，作 log 记录
```

```
UINT nCurPos= DEM_GetAxisLmtZeroState (0); //当前 X 轴信号状态
```

6.8.1.2 零位限制负向

bool **__stdcall** DEM_EnableZeroLimitNegMov(**bool** bEnable, **unsigned int** nAxisMask);

参数

bEnable

true 零位开启负限位功能

false 零位关闭负限位功能

nAxisMask

指定轴掩码, 0x1 X轴 0x2 Y轴

返回值

TRUE 正在运动

FALSE 运动已停止

示例

```
//用来与运动系统的位置进行对比, 检验编码系统的准确性, 作 log 记录  
  
UINT nRuning= DEM_EnableZeroLimitNegMov(TRUE,0x1);    //当前 X 轴信号  
状态
```

6.8.1.3 限位使能

bool **__stdcall** DEM_EnableLimitFun(**unsigned int** nAxis, **bool** bEnable);

参数

nAxis

指定轴索引号

bEnable

true 为使能限位功能, false 为不是能限位功能

返回值

true 轴正在运动

False 运动已停止

示例

```
//用来与运动系统的位置进行对比, 检验编码系统的准确性, 作 log 记录  
  
UINT nRuning= DEM_EnableLimitFun(0,TRUE); //使能指定轴正负限位功能  
//即正向运动时, 当正限位被触发, 正向运动将被限制住, 负向同理
```

6.8.1.4 轴编码初始化

void __stdcall DEM_SetAxisEncoderValue(unsigned int nAxis,unsigned int nEncVal);

参数

nAxis

指定轴索引号

nEncVal

指定编码器值

返回值

void

示例

```
//用来与运动系统的位置进行对比，检验编码系统的准确性，作 log 记录  
  
DEM_SetAxisEncoderValue(0,0x100000); //指定光栅当前计数值为 0x100000  
//指定初始值的目的在于定位运动
```

6.8.1.5 零位复位计数

bool __stdcall DEM_EnableZeroResetEncVal(unsigned int nAxis, bool bEnable);

参数

nAxis

指定轴索引号, 0 : X轴 1 : Y轴 2 : Z轴

bEnable

true 复位指定轴为初始值。

false 不复位指定轴为初始值。

返回值

true 指定轴复位成功。

false 指定轴复位失败。

示例

```
//用来复位运动系统初始值  
  
DEM_EnableZeroResetEncVal (0,TRUE); //指定轴零位触发时，复位计数值  
//用于运动系统初始化过程
```

6.8.2 双 Y 轴控制

6.8.2.1 端口使能

bool **__stdcall** DEM_EnableDYOutPut(**bool** bY1Out, **bool** bY2Out);

参数

bY1Out
true使能，false不使能
bY2Out
true使能，false不使能

返回值

TRUE 执行成功
FALSE 执行失败

示例

```
//控制双 Y 轴运动时的使能控制  
  
DEM_EnableDYOutPut (TRUE,FALSE); //当前 Y1 输出 Y2 不输出
```

6.8.2.2 设置同步误差

bool **__stdcall** DEM_SetDYSyncCtlMaxOff(**unsigned int** nOffset);

参数

nOffset
设置 Y1 与 Y2 轴的最大同步偏移量，uint：编码值

返回值

TRUE 执行成功
FALSE 执行失败

示例

```
//控制双 Y 轴运动时，设置最大同步偏差  
  
DEM_SetDYSyncCtlMaxOff (100); //Y1 和 Y2 最大同步偏差
```

6.8.2.3 查询同步误差

unsigned int **__stdcall** DEM_QueryDYSyncOffset();

参数

void

查询 Y1 轴与 Y2 轴的同步偏移量

返回值

unsigned int

Y1 轴与 Y2 轴的偏移量

示例

//控制双 Y 轴运动时，查询最大同步偏差

UINT nCurOffset=DEM_QueryDYSyncCtlOffset (); //查询 Y1 和 Y2 同步偏差

6.8.2.4 查询同步状态

bool **__stdcall** DEM_QueryDYSyncError();

参数

void

返回值

true 查询双Y有错误。

false 查询双Y无错误。

示例

//控制双 Y 轴运动时，查询最大同步偏差

BOOL bError=DEM_QueryDYSyncError (); //查询 Y1 和 Y2 同步状态

6.8.3 运动使能

bool **__stdcall** DEM_EnableAxisRun(**bool** bEnable);

参数

bEnable

轴电机使能输出，TRUE 使能，FALSE 不使能

返回值

TRUE 执行成功

FALSE 执行失败

示例

```
//所有轴运动前的一个使能控制，执行一次即可
```

```
DEM_EnableAxisRun(TRUE); //在执行运动启动前执行一次即可
```

6.8.4 运动初始化

bool **__stdcall** DEM_InitAxis(**unsigned int** nAxis, **unsigned int** nCtlFlag);

参数

nAxis

指定轴索引号

nCtlFlag[in]

[15:0] 加速度 Uint：脉冲/毫秒平方

16b 输出方向反转

17b 限位连接反

18b 编码器计数反

19b 零位有效信号反

20b 正限位有效信号反

21b 负限位有效信号反

22b 输出方式[0:方向+脉冲 1:双向脉冲]

23b 零位信号源采用OPD

24b 车头Z轴编码器采用指令脉冲计数

返回值

TRUE 执行成功

FALSE 执行失败

示例

```
//用来与运动系统的位置进行对比，检验编码系统的准确性，作 log 记录
```

```
bool bInitied= DEM_InitAxis (0,0x100); //初始化 X 轴，加速度 256 信号
```

6.8.5 运动启动

```
bool __stdcall DEM_Run(unsigned int nAxis, bool bDir, unsigned int nPlsSpd,  
int nPlsCnt, unsigned int nCtrFlag);
```

参数

nAxis[in]
指定轴索引号

bDir [in]
运动方向, FALSE 正向, TRUE 负向

nPlsSpd [in]
脉冲速度

nPlsCnt [in]
总的脉冲数

nCtrFlag [in]
bit[0] 0 等停, 1 不等停
bit[1] 双Y轴运动时, 1 双Y同步 0 双Y不同步

返回值

TRUE 执行成功
FALSE 执行失败

示例

```
//用来控制指定轴的运动启动控制  
int nYDPI=100; // 喷头物理 DPI  
int nXDPI=25400; // 1um 光栅  
float fSpeed=100.0f; //100 mm/s  
int nXSpeed=MM_TO_DOT(100,nXDPI); //换算成脉冲  
int nYSpeed=MM_TO_DOT(100,nYDPI)*48; // 48 为 Y 运动单位  
int nXStep=MM_TO_DOT(100,nXDPI); //换算成脉冲  
int nYStep=MM_TO_DOT(100,nYDPI)*48; // 48 为 Y 运动单位  
bool bXStart= DEM_Run (0,FALSE,nXSpeed,nXStep,FALSE); //运动 X 轴
```

6.8.6 运动状态

bool **__stdcall** DEM_AxisIsRuning(**unsigned int** nAxis, **bool*** pDir, **unsigned int*** pRPos);

参数

nAxis[in]
指定轴索引号
pDir [in]
运动方向
pRPos [in]
运动位置值

返回值

TRUE 轴正在运动
FALSE 运动已停止

示例

```
//用来控制指定轴的运动启动控制
bool bDir=FALSE; //运动方向
UINT nRevPls=0; //剩余脉冲数
while(DEM_AxisIsRuning(0,&bDir,&nRevPls)) //等待 X 轴停止
{
    Sleep(1);
}
```

6.8.7 运动停止

bool **__stdcall** DEM_Stop (**bool** blmmeStop, **unsigned int** nAxisMask);

参数

blmmeStop [in]
为true时，紧急停止，否则 带减速停止
nAxisMask [in]
指定轴掩码:
bit[0] X轴
bit[1] Y轴
bit[2] Z轴

返回值

TRUE 执行成功
FALSE 执行失败

示例

```
//用来停止当前运动轴
bool blnited= DEM_Stop (FALSE,0x1); //停止 X 轴运动，可同时停止多轴运动
```

6.9 ADIB 控制

6.9.1 负压和温度设置和回读

bool __stdcall DEV_AdibControl(LPADIB_PARAM lParam,UINT nOption,BOOL bSetParam,BOOL* bAbort)

参数

lParam

是 ADIB 全部参数描述的结构体，详见附录；

nOption

需要操作 ADIB 的功能项,0x1 版本号 0x2 温度 0x4 负压

bSetParam

ADIB 的读写控制， FALSE 读 TRUE 写

bAbort

中止当前操作

返回值

无

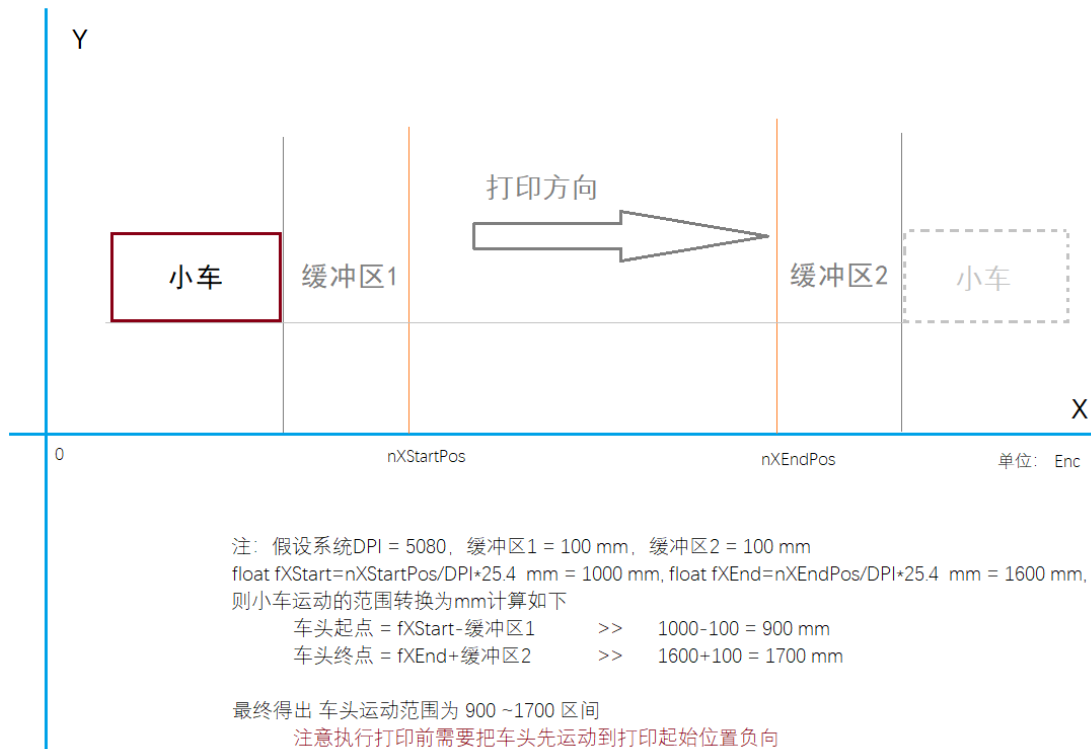
示例

```
//用来控制外为设备的温度
ADIB_PARAM adibParam;
UINT nOption=0x6;
DEV_AdibControl(&adibParam,nOption,FALSE,NULL); //获取ADIB信息

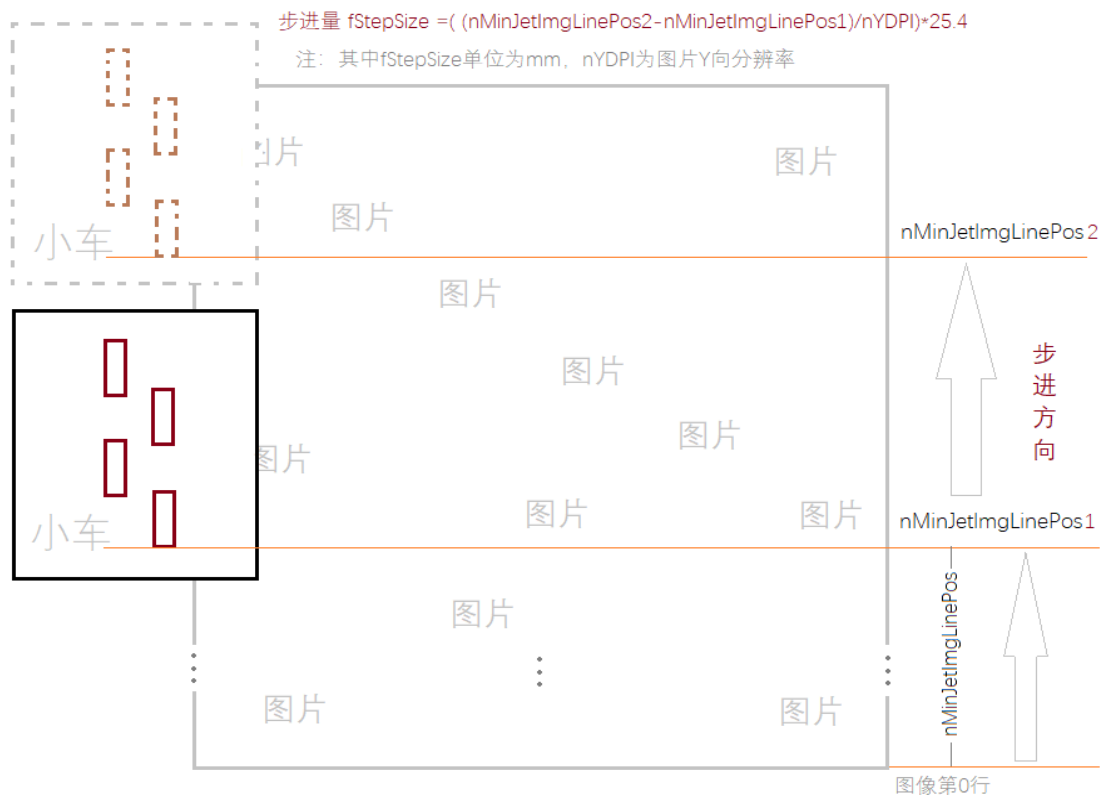
ADIB_PARAM adibParam;
UINT nOption=0x6;
for(int i=0;i<8;i++)
{
    adibParam.fcurAirPress[i]=-5.0; //kpa
    adibParam.fcurlnkTankTemp[i]=30.0f; //°C
}
DEV_AdibControl(&adibParam,nOption,TRUE,NULL); //设置ADIB信息
```

七、系统参数的计算

7.1 X 运动范围计算



7.2 Y 步进量计算



7.3 常用计算公式

7.3.1 mm 转 dot

```
#define MM_TO_DOT(X,DPI)  (int)((((float)(X*DPI))/25.4+0.45f))
```

7.3.2 dot 转 mm

```
static float DOT_TO_MM (int X,float DPI)
{
    //ASSERT(DPI!=0);
    int  nresult;
    float nl=0.005f;
    if(X<0)
        nl=-0.005f;
    nresult= (int)(10*(((float)X*25.4)/DPI+nl));
    return ((float)nresult)/10;
}
```

7.3.3 栅距 μ 转DPI

先将栅距转换为mm单位，则根据DPI定义计算如下

```
#define U_TO_DPI ( $\mu$ )  (25.4/( $\mu$ ))
```

7.3.4 打印分频值

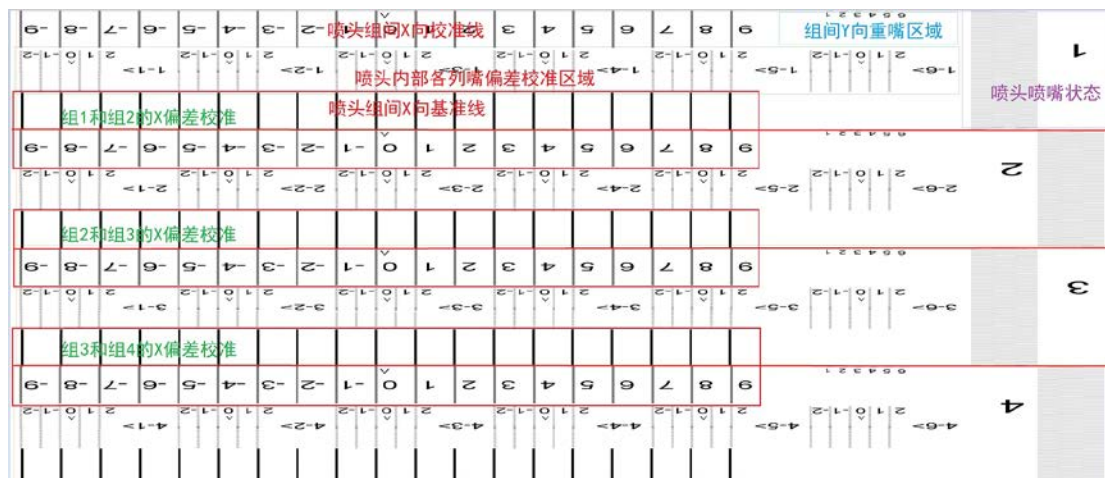
打印图像X的分辨率为nXDPI，系统光栅为nSysDPI计算如下

分频值 = nSysDPI/nXDPI 所以打印图像的XDPI一般能整除光栅DPI，这样精度更高

八、喷头校准说明

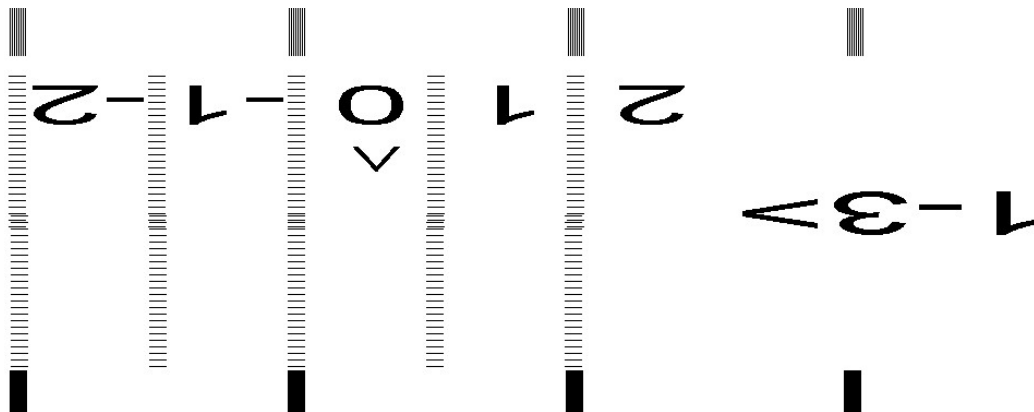
校准图都是游标卡尺原理，假设基准线间距是 100 像素，则校准线间距是 99 像素

8.1 X 向偏差校准图



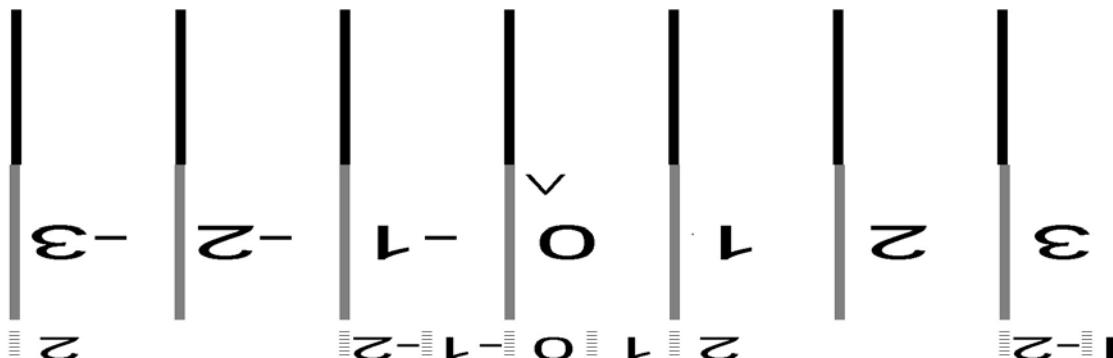
图中彩色线框和文字仅为作说明,并非原图数据,系统打印的原始数据为单单位图数据,打印此校准时不需要产生 Y 向步进,需要上位机开发者注意的是,所有校准的偏差值最终都需要转换为相对组 1 第一列喷嘴的结果。(以组 2 的 2-3 为例,假设组 1 和组 2 的偏差值是 5, 2-3 的值是 3, 那么 2-3 相对组 1 第一列的偏差值为 $5+3=8$)

8.1.1 喷头内部 X 校准区域



图示为 喷嘴第 3 列和基准列第 1 列的 X 单向的偏差

8.1.2 喷头组间 X 校准区域

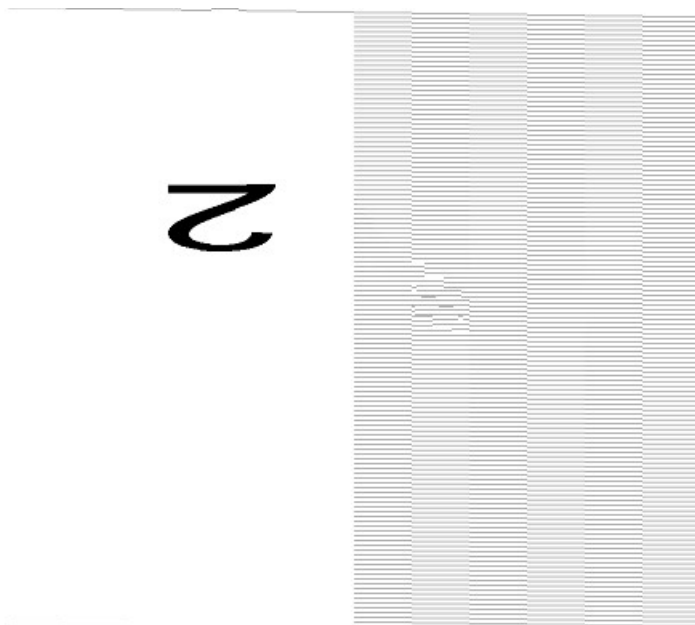


8.1.3 喷头组间 Y 校准区域



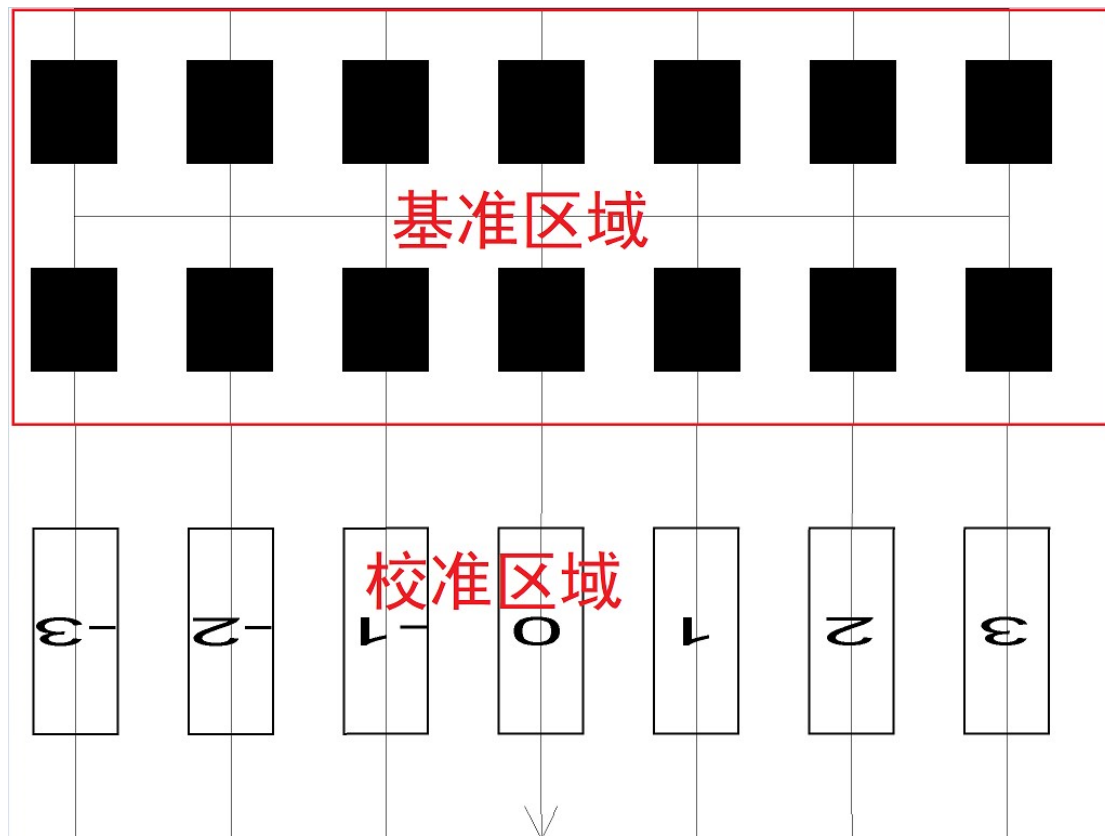
观察喷头组Y向喷嘴重叠情况

8.1.3 喷头喷嘴状态区域



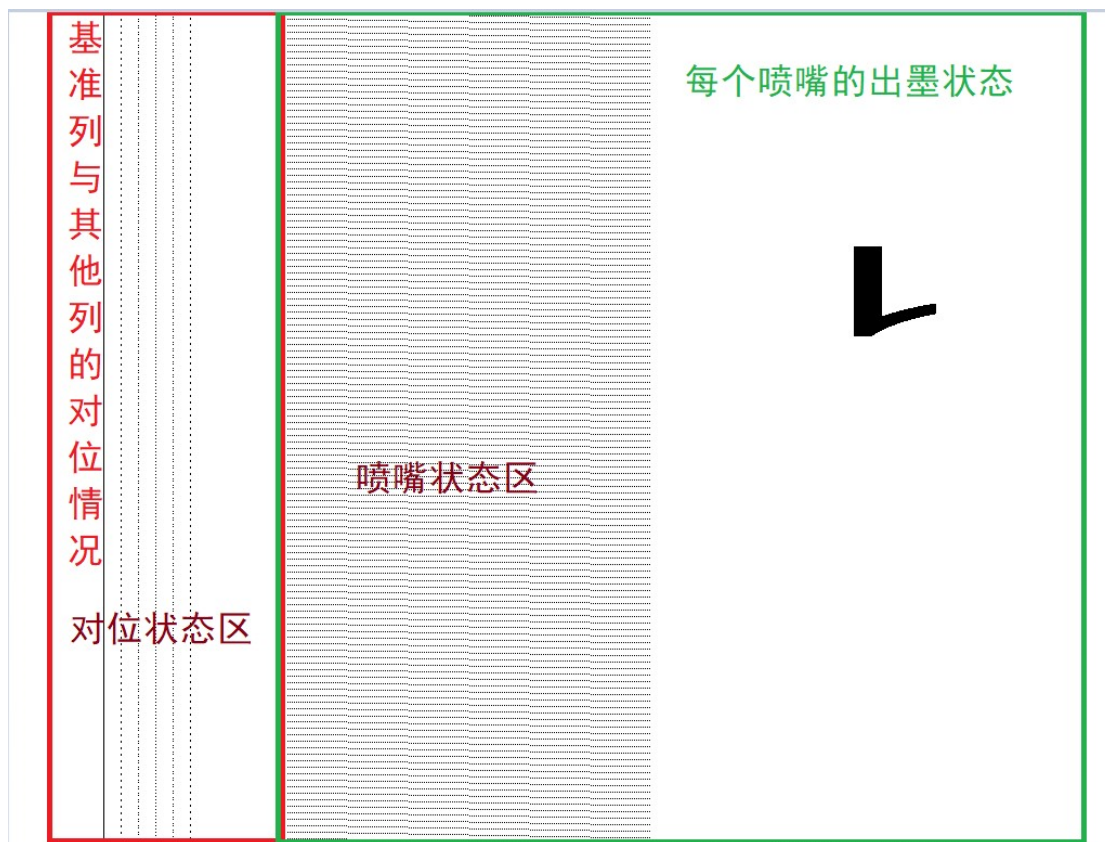
图示为组2喷头的各个喷嘴出墨状况

8.2 往返差校准图



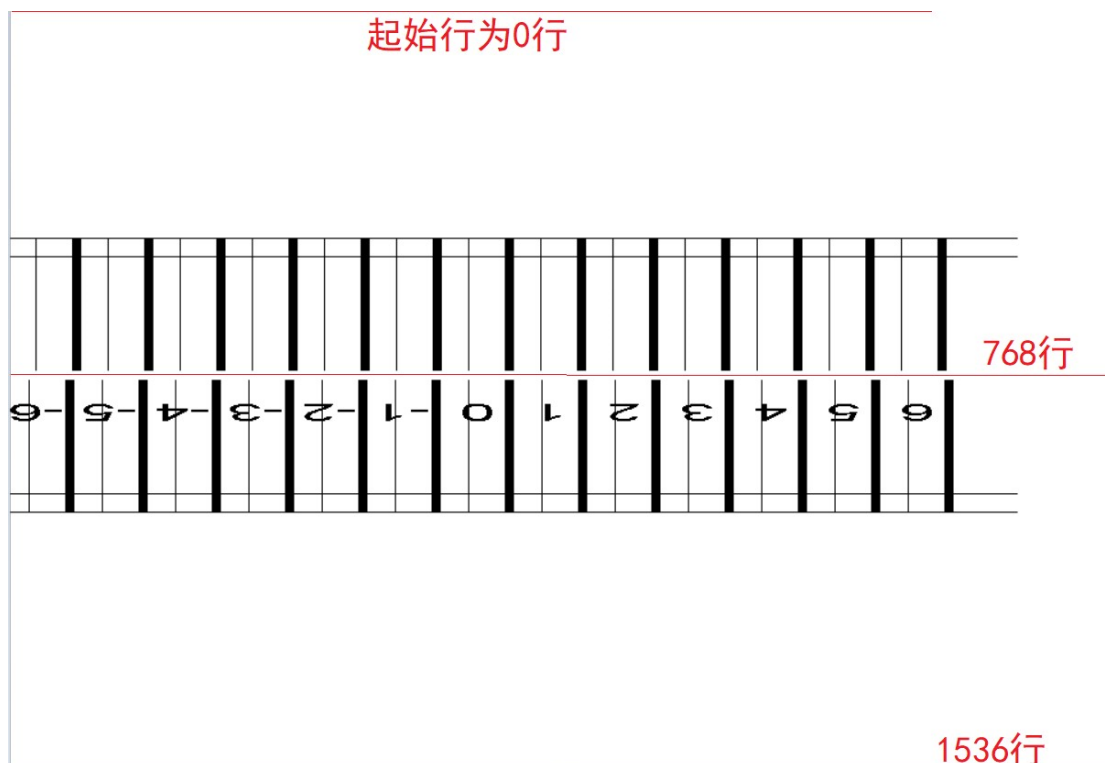
往返差校准可以通过设置打印图层信息 `PRTIMG_LAYER` 的 `nPrtFlag` 的 `bit[2]` 置1来控制 2PASS双向打印，第1pass打出基准区域（正向打印），第2pass打印校准区域（反向打印），注意打印此校准时不需要产生步进，注意的是所有校准打印时 `PRTJOB_ITEM` 中 `nPrtCtl` 的 `bit[2]` 需要置1

8.3 喷嘴状态图



图示从左至右 对位状态区 中实线为6列嘴的对位状态，虚线从左至右分别是 1-2、1-3、1-4、1-5、1-6的对位状态，喷嘴状态区为查看喷头喷嘴情况的，Y向有多少组头，就有多少个这样的状态，喷嘴状态区的短线呈台阶状

8.4 喷头机械安装



该部分校准原理很简单，只是为了检测喷头组安装的垂直情况，图中数值没实际意义，只能物理调节，需要微动Y步进来完成打印，图示为一组头的打印图，图片需要2PASS数据行，开发者可自己设计。

SDK附带有校准图生成源码工程，仅供参考

九、PHMAN 工具说明

PHMAN工具是用来描述喷头在虚拟坐标轴里面的物理位置关系是影响打印数据的最关键最直接参数配置工具，坐标轴的定义其参照上下文，通道号是按照驱动卡ID一次递增，下文会有说明，该部分仅介绍与该系统开发相关参数工具界面介绍如下：

Config-0

将Data Lines 补偿为8的整数倍

Groups: 4
PH Physical DPI: 300
Split Count: 6
Jets Per Group: 768
Combo Type: 交叉组合
Data Line Offset: 0
Data Lines: 128
Spot Color count: 0
Jet Rate: 1

喷头首尾相接的组数
喷头组Y向物理分辨率
喷头组喷嘴列数
喷头组喷嘴数
喷头并排的个数
喷头一列嘴的数据行数
该值需要为1

Short Set
Gen Calibration File
PreView

Std Color Offset
物理位置描述
Channel Option
喷头通道号

Color Tbl	C
Y Jet Off	0.00
S1	43.00
S2	43.00
S3	32.00
S4	43.00
S5	43.00
S6	32.00
Y Jet Off	728.00
S1	18.00
S2	8.00
S3	0.00
S4	18.00
S5	8.00
S6	0.00
Y Jet Off	1636.00
S1	43.00
S2	43.00
S3	32.00
S4	43.00

注：图示仅标记与QS256系统开发相关的参数配置，Vertical Data Reverse 需要说明的是，该参数由 喷头实际安装后喷头的0号嘴是否为Y零点或者靠近Y零点方向，如果不符合，则需要勾选，同时对应的物理位置需要左右翻转

Support Pass Table

Index	Pass	Comment
1	1	
2	2	
3	3	
4	4	
5	5	
6	6	
7	7	
8	8	
9	9	
10	10	
11	11	
12	12	
13	13	
14	14	
15	15	
16	16	
17	17	
18	18	
19	19	
20	20	
21	21	
22	22	
23	23	
24	24	

Comment:

Base Option

Print Encoder DPI: 5080
Device Code: 0
Application Type ID: 0
Control Groups: 1
Phy Data Channel Count: 192
PH Count: 24
Max fire frequency: 40000
Gray Fire Frequency: 20000
PH Fire Pulse Width: 100
Calibration File DPI: 635
Calibration Rows Unit: 1
Ink Supply Count: 1
PrintHead type: QS256
Machine type: Polt
Encoder Source: Printer Boa

Vertical Data Reverse
喷头数据垂直翻转

Carriage Home In Right Side
车头条点在右边

Support Roll And FlatBed
Lifter Back Print Height

Print Phase Reverse
Spot Color Half Fire
UV Lamp Control
Clean Valid
Media Detection
Step Adjust By Print Length

Enable Carriage Lift
Lifter Back Print Height
Vertical Data Reverse
Group Indecency Print
Enable Double Print
Capping Valid
Media Exception Monitor
Enable PLC Comm

Color Tank Lookup Table

9.1 驱动卡通道号

驱动卡通道号按照驱动卡ID，因为QS256模组有6列嘴，所以通道号每6个一组递增，例如：

驱动卡1 通道号对应0~5、

驱动卡2 通道号对应6~11、

驱动卡3 通道号对应12~17、

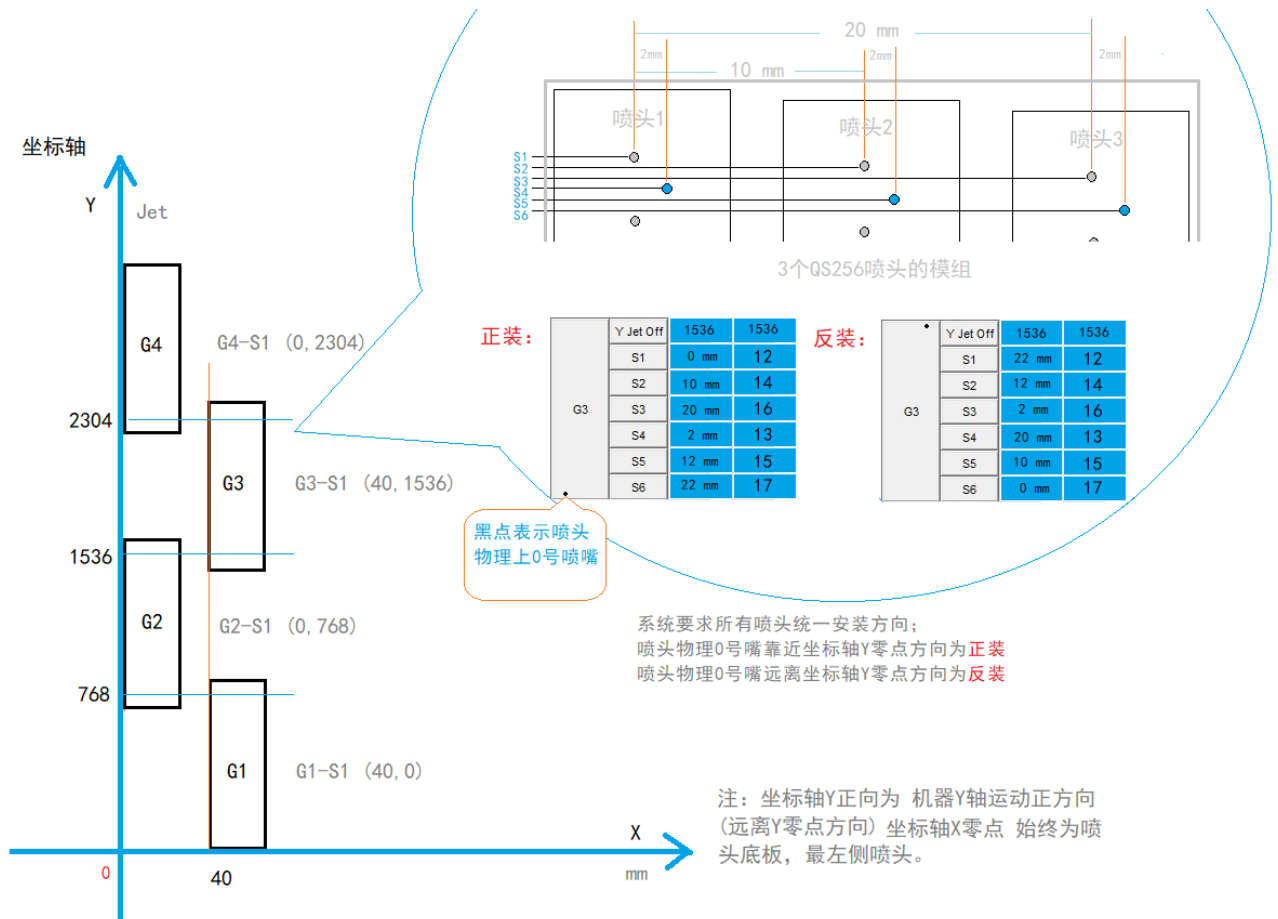
驱动卡4 通道号对应18~23

驱动卡ID定义请参照前文 [四、系统卡ID定义](#)

9.2 喷头位置参数

喷头安装物理上可分为正装和反装，**正装**即为喷头物理上0号喷嘴与坐标系Y轴零点方向一致，否则为**反装**，当前系统要求全部喷头必须统一安装方向，选择正装或者反装，X方向要保证远离X零点方向编码计数增大，坐标轴X零点与车头在左或者右无关。（坐标轴的建立前参照前文 [三、坐标轴的定义](#)）

安装参数的关系如下图：



需要注意的是，如果运动系统中小车停泊**原点在右**需要勾选 **Carriage Home In Rightside**

如果此选项没有正确配置，会导致打图X镜像，偏差校准参数反向

喷头Y向**反装**时需要勾选 **Vertical Data Reverse**，同时喷嘴列的X向位置参数需要反转，如果此配置不匹配，导致打印数据错乱，Y向无法对位

十、附录

具体信息结构体请参照提供的接口文件为准,以下仅供参考。

PRTIMG_LAYER,*LPPRTIMG_LAYER

typedef struct tag_ImgLayer

```
{
    long    nLayerIndex;        //图层索引
    int     nImgStartJetIndex;  //图像起始对应喷头嘴;
    int     nPrtDir;            /*图像打印 X 方向,多 PASS 时的起始反向
                                0 编码负方向 1 编码正方向 */
    int     nXEncOff;           //图层针对任务 X 起点偏移 单位 X 编码
    int     nYJetOff;           //图层针对任务 Y 起点偏移 单位 喷嘴间距
    int     nColorCnts;         //颜色通道数量
    long    nXDPI;              //图像 XIDPI
    long    nYDPI;              //图像 YIDPI 计算时暂时不用,默认为喷头组 DPI
    long    nBytesPerLine;      //每行数据字节数
    long    nHeight;            //图像高度 单位: 像素
    long    nWidth;             //图像宽度 单位: 像素
    long    nPrtFlag;           //bit0 双向打印 bit1: Y 打印方向 1 反向打印 bit2 往返差打印
    long    nReserved[8];
}PRTIMG_LAYER,*LPPRTIMG_LAYER;
```

PassDataItem,*LPPassDataItem

typedef struct tag_PassDataItem

```
{
    unsigned int  nLayerIndex;        //所属图层号
    unsigned int  nLayerPassCount;    //所属图层 PASS 总数
    unsigned int  nLayerPassIndex;    //PASS 序号 0 为第一 PASS
    unsigned int  nValidPassJets;     //PASS有效喷嘴数 20180818
    unsigned int  nProcState;         /*图层处理状态
                                0 数据处理阶段
                                1 数据处理就绪
                                2 写数据到硬件
                                3 数据已写入硬件
                                4 已添加到打印队列
                                5 打印已结束 */
    int           nMinJet0ImgLinePos; //Y 向偏差最小喷头组的 0 号喷嘴相对图像的行位置
    Bool          bPrtDir;            //打印方向 0 编码负方向 1 正方向
    unsigned int  nValidPrtCtlCnts;    //涉及到的打印控制器数量
    unsigned int  nValidPrtCols;       //有效打印列数,可理解为像素
    unsigned int  nStartEncPos;        //起始位置 编码值
    unsigned int  nPrtPrecession;      //打印分频值 多少光栅点喷墨一次
}
```

```
unsigned int    nPrtMemHwAddr[MAX_PRTCTL_CNT];    //板卡物理内存地址 nBytes
unsigned int    nPrtDataOffset[MAX_PRTCTL_CNT];    //数据起始偏移
unsigned int    nSrcDataSize[MAX_PRTCTL_CNT];    //数据大小
unsigned int    nPrtColBytes[MAX_PRTCTL_CNT];    //列长度 1PASS 数据字节数
unsigned int    nHwMemAdrMatchMask;    //已分配打印内存的 MASK
int            nDataTxCompleteCnt;    //传输完成的次数
int            nSrcStartCols;    //打印起始列, 相对打印起点偏移的像素
int            nSrcEndCols;    //打印结束列, 相对打印起点偏移的像素
unsigned char*  pDataBuf;    //目标数据缓存区
tag_PassDataItem* pNextItem;
}PassDataItem,*LPPassDataItem;
```

PrtRunInfo,*LPPrtRunInfo

```
typedef struct tag_PrtRunInfo
```

```
{
    bool        bJobPrtRuning;    //作业打印运行中
    bool        bLayerPrtIsOver;    //当前打印图层全部 PASS 打印完成
    int         nPrtState;    //打印控制器状态 0 待机 1 打印 2 闪喷 3 暂停 4 ERROR
    int         nPrintLayerIndex;    //当前打印的图层
    int         nLayerPassCount;    //当前打印的图层 PASS 总数
    int         nPrintPassIndex;    //当前打印的 PASS
    int         nCurPrtDir;    //当前的打印方向 0 和 1
    int         nRevPrtCols;    //当前剩余的打印列数
    int         nProcLayerIndex;    //当前数据处理的图层
    int         nDTLayerIndex;    //当前数据传输的图层
    int         nDTLayerPassIndex;    //当前数据传输的 PASS
    int         nDTPtrCtlIndex;    //当前数据传输的控制器 0:车头卡 1 1:车头卡 2
    int         nContWDErr;    //连续写内存数据错误
    int         nContReqMemErr;    //连续申请内存失败计数
}PrtRunInfo,*LPPrtRunInfo;
```

PRTJOB_ITEM,*LPPRTJOB_ITEM

```
typedef struct tag_PrtJobItem
```

```
{
    unsigned int nJobID;    //作业号
    char        szJobName[64];    //作业名
    unsigned int nPrtXEncPos;    //任务的 X 向起打位置
    float        fPrtYPos;    //保留 离 0 位位置
    long         nPixelGrayBits;    //灰度数据位数
    unsigned int nPrtCtl;    /*bit0 跳白支持
                                bit1 循环喷嘴偏移
                                bit2 Y 向偏差无重嘴
                                bit4 X 镜像
                                bit5 Y 镜像 */
}
```

```
}PRTJOB_ITEM,*LPPRTJOB_ITEM;
```

RYSYS_PARAM,*LPRYSYS_PARAM

```
typedef struct tag_RYSYS_PARAM
```

```
{
    long nParamVer;           //参数版本
    long nParamSize;          //参数字节数
    int    nBiDirEncPrtOff;    // 双向偏差 校准图数值*打印分频值
    int    nPhXRowPrtOff[MAX_DRV_CNT][MAX_JETROW][DIR_COUNT]; /*单向偏差
                                                                校准图数值*打印分频值
                                                                0 负方向 1 正反向 [喷头号][喷嘴列号][方向] */
    int    nPhYJetOff[MAX_COLORS][MAX_GROUP]; //Y 向嘴偏差
    DRV_WAVEFORM drvWaveForm[MAX_DRV_CNT];    //驱动卡温度电压
    unsigned int nMicroJetUint;                //PASS 微动时变化单位 Jet
    unsigned int nMicroJetCount;               //PASS 微动嘴 Jet
    int    nOverlapJetProcType;                /* 重叠嘴处理方式
                                                                0 交叉 1 后喷头舍弃与前喷头重叠部分*/
    float fBurstCycleSec;                      //闪喷的周期 秒
    float fBurstValidSec;                      //闪喷的有效时间 秒
    float fBurstFrequency;                     //闪喷频率 Hz
    int    nPrtEncDir;                        //编码计数方向 0 正计数 1 反计数
    char    szLogPath[MAX_PATH];              //日志文件路径
    unsigned int nResMemory;                   //保留物理内存的大小 单位 MB
    unsigned int nRevParam[127];              //保留参数
}RYSYS_PARAM,*LPRYSYS_PARAM;
```

PRINTER_INFO,*LPPRINTER_INFO

```
typedef struct tag_Printer_Info
```

```
{
    unsigned int nVersion;                //打印机软件系统版本号
    unsigned int nCustomlerD;             //客户号
    unsigned int nSystemXDPI;             //系统光栅 DPI
    unsigned int nMainFpgaVer;            //主控板 FPGA 版本
    unsigned int nCarFpgaVer[4];          //车头板 FPGA 版本
    unsigned int nMcuVersion[4];          //车头卡 MCU 版本
    unsigned int nPrtValidMask;           //车头卡连接掩码
    unsigned int nDrvValidMask[4];        //驱动卡连接掩码
    unsigned int nDrvWaveErrMask[4];      //驱动卡波形加载失败掩码
    unsigned int nUsbVersion;             //USB 固件版本
    unsigned int nStatus;                 //系统状态
    unsigned int nPrintStatus;            //0 待机 1 打印 2 暂停
    unsigned int bSuperDevice;            //U30 设备
    DRVINFO sysDrvInfo[MAX_DRV_CNT];      //驱动卡状态信息
    PprtRunInfo prt_rinfo;                //打印运行状态
}
```

```
unsigned int    nLastErrClrModule; //Y 向偏差过大, 导致组间分离的状态 颜色索引
unsigned int    nLastErrPhgIndex;  //Y 向偏差过大, 导致组间分离的状态 组索引
unsigned int    nResMemory;        //保留内存区的大小 单位: MB 兆字节
                                   限定最小值 256 MB 20180827

unsigned int    nRevInfo[127];
unsigned int    nCrc32;
}PRINTER_INFO,*LPPRINTER_INFO;
```

DRV_WAVEFORM,*LPDRV_WAVEFORM

```
typedef struct tag_DRV_WAVEFORM
```

```
{
    UINT        nPlsNum;           //脉冲数量
    float        fPluseT2;         //脉冲保护间隔 单位 1us
    float        fHPluseGap[3][3]; //多脉冲间隔 单位 1us
    UINT        fHPluseWidth[4][3]; //高压脉宽 单位 1us 4*3 个
    float        fHVdestVoltage[3]; //电压设置值 单位 1us 3*3 个
    float        fBoardAlarmTemp;  //板卡报警温度 单位 °C
    UINT        nRevParam[2];      //保留
    char        szName[32];        //波形名称
}DRV_WAVEFORM,*LPDRV_WAVEFORM;
```

DRVINFO,*LPDRVINFO

```
typedef struct tag_DRVINFO
```

```
{
    unsigned int    nState;         //连接状态 1 表示已连接
    unsigned int    nNextState;     //连级下个驱动卡连接状态
    unsigned int    nSignature;     //驱动卡序列号
    unsigned int    nPCBVersion;    //驱动卡 PCB 版本
    unsigned int    nFMVersion;     //驱动卡 MCU 版本
    unsigned int    nFpgaVersion;   //驱动卡控制程序版本
    unsigned int    nPtvwarnState;  //1 喷头不存在 2,8 喷头温度超过控制范围
    unsigned int    nRevInfo[32];
    unsigned int    nCrc32;
}DRVINFO,*LPDRVINFO;
```

ADIB_PARAM,*LPADIB_PARAM

```
typedef struct tag_ADIB_PARAM //20170907
```

```
{
    UINT    nFMver;           //程序版本号
    float    fcurvoltage[8];  //当前电压
    float    fcurInkTankTemp[8]; //当前温度
    float    fcurAirPress[8]; //当前负压
    UINT    nLgStatus;        //保留输入
    UINT    nVolState;        //当前电压状态
}
```

```
Struct I2C_DATA{float fvoltage[8];float fInkTankTemp[8];float fAirPress[8];}I2C; //I2C 保  
存的数据  
UINT nReverse[2]; //预留  
}ADIB_PARAM,*LPADIB_PARAM;
```